IES SAN ANDRÉS

ESPECIALIZACIÓN EN INTELIGENCIA ARTIFICIAL Y BIG DATA

# SISTEMAS DE BIG DATA

# UT04: ANÁLISIS Y BÚSQUEDA DE RESPUESTAS EN DATOS

# ÍNDICE

1.- Introducción

2.- Metodologías de minería de datos: SEMMA

3.- Metodologías de minería de datos: CRISP-DM

# 2 LA METODOLOGÍA SEMMA

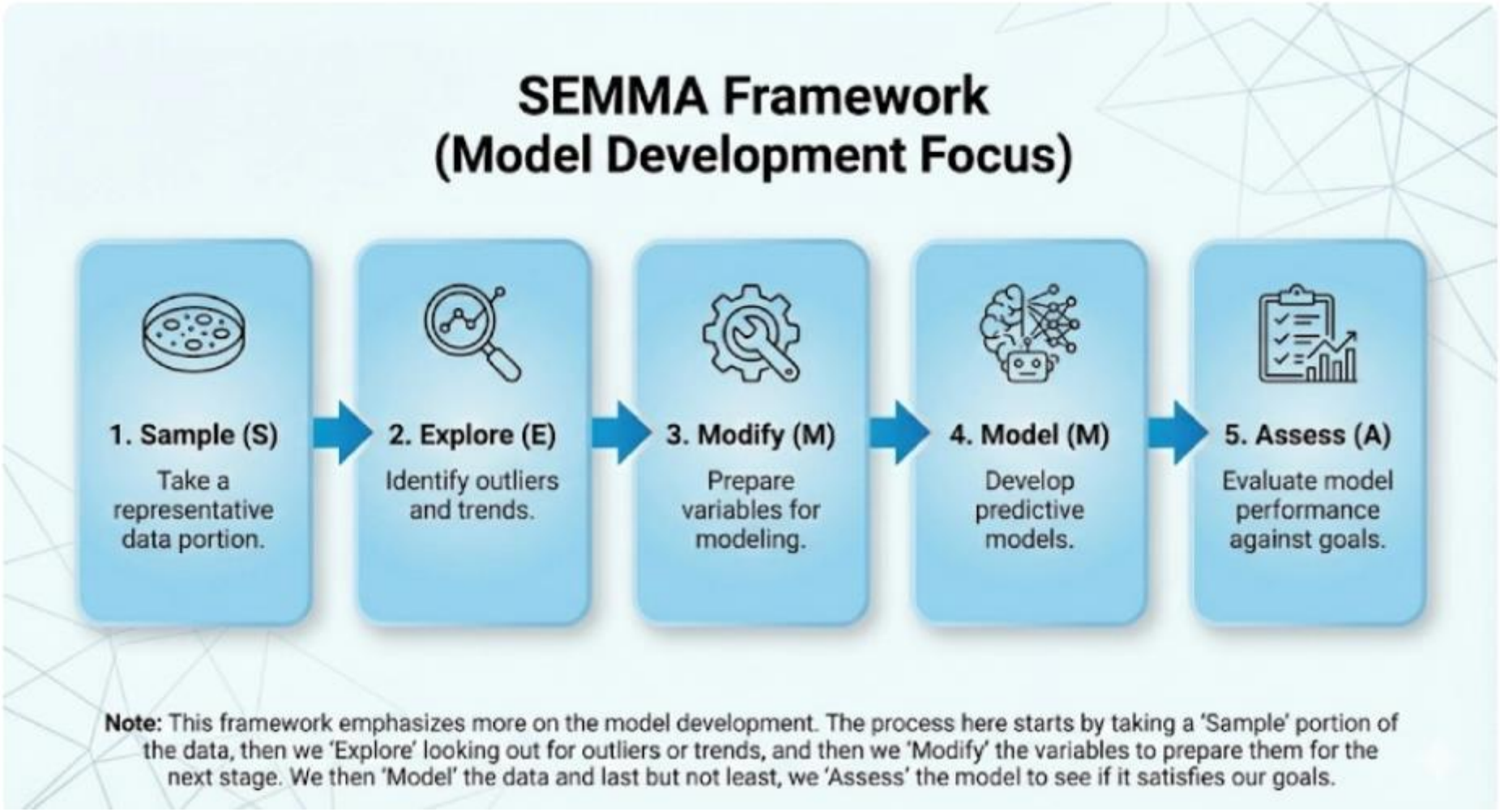
# 2 LA METODOLOGÍA SEMMA

## 2.1 INTRODUCCIÓN A SEMMA

La metodología **SEMMA (*Sample, Explore, Modify, Model, Assess*)** fue desarrollada por el SAS Institute y es la preferida cuando el enfoque es puramente **experimental y científico**.

Es una metodología que se caracteriza por ser muy **cíclica** y estar centrada en el **refinamiento del modelo**.

Esta tubería se visualiza como una tubería de refinamiento donde **empieza con datos brutos y termina con conocimiento evaluado**


SEMMA Framework
(Model Development Focus)
1. Sample (S)
Take a representative data portion.
2. Explore (E)
Identify outliers and trends.
3. Modify (M)
Prepare variables for modeling.
4. Model (M)
Develop predictive models.
5. Assess (A)
Evaluate model performance against goals.
Note: This framework emphasizes more on the model development. The process here starts by taking a 'Sample' portion of the data, then we 'Explore' looking out for outliers or trends, and then we 'Modify' the variables to prepare them for the next stage. We then 'Model' the data and last but not least, we 'Assess' the model to see if it satisfies our goals.

# 2 LA METODOLOGÍA SEMMA

## 2.2 LA FASE SAMPLE DE SEMMA

Todos los ejemplos de este apartado están aplicados sobre este dataset.

## The Movie Database (TMDB) Comprehensive Dataset

10,000 Movies with 150k Cast, 63k Crew, and 80k User Reviews

```python
import pandas as pd
from sklearn.model_selection import train_test_split

try:
    df = pd.read_csv('movies.csv', on_bad_lines='warn', engine='python')
except Exception as e:
    print(f"Error al cargar: {e}")

df.head(2)
```

✓ 0.1s

# FASE SAMPLE (MUESTREO)

El objetivo de esta fase no es solo dividir los datos, sino **asegurar que la muestra sea representativa** de la realidad futura.

Algunos conceptos clave de esta fase son:

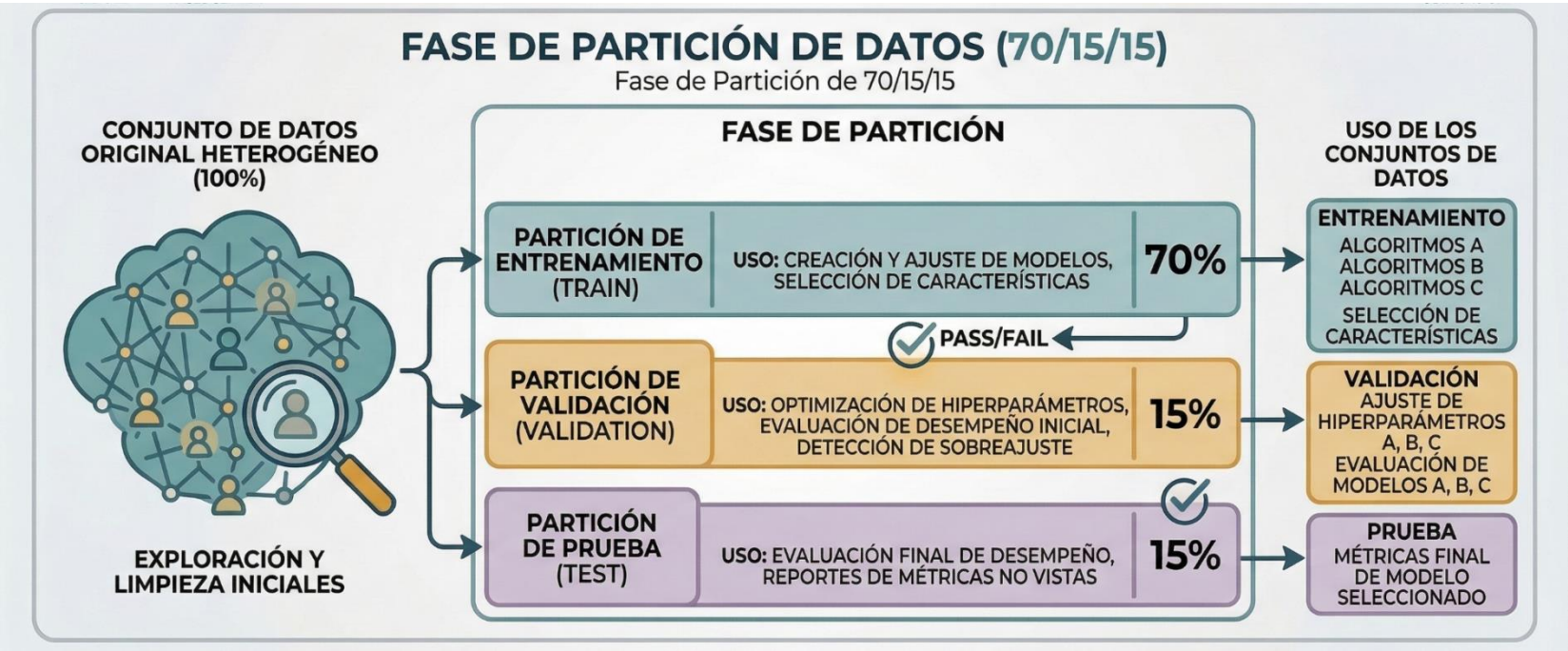
- Partición 70/15/15
- Muestreo estratificado
- Validación cruzada

## Partición 70/15/15

Es habitual dividir los datos en tres grupos:

- **70% para entrenamiento**: utilizados para que el algoritmo busque patrones, correlaciones y reglas.
- **15% para validación**: se utiliza para ajustar hiperparámetros y evitar el sobreajuste. P.e., si usas 3 algoritmos diferentes, se usan los datos de validación para ver cuál funciona mejor.
- **15% para test**: proporciona la evaluación final e imparcial de la precisión del modelo.

SAMPLE
MUESTREO
EXPLORE
EXPLORACIÓN
MODIFY
MODIFICACIÓN
MODEL
MODELADO
ASSES
EVALUACIÓN


FASE DE PARTICIÓN DE DATOS (70/15/15)
Fase de Partición de 70/15/15
CONJUNTO DE DATOS ORIGINAL HETEROGÉNEO (100%)
EXPLORACIÓN Y LIMPIEZA INICIALES
FASE DE PARTICIÓN
PARTICIÓN DE ENTRENAMIENTO (TRAIN)
USO: CREACIÓN Y AJUSTE DE MODELOS, SELECCIÓN DE CARACTERÍSTICAS
70%
PASS/FAIL
PARTICIÓN DE VALIDACIÓN (VALIDATION)
USO: OPTIMIZACIÓN DE HIPERPARÁMETROS, EVALUACIÓN DE DESEMPEÑO INICIAL, DETECCIÓN DE SOBREAJUSTE
15%
PARTICIÓN DE PRUEBA (TEST)
USO: EVALUACIÓN FINAL DE DESEMPEÑO, REPORTES DE MÉTRICAS NO VISTAS
15%
USO DE LOS CONJUNTOS DE DATOS
ENTRENAMIENTO
ALGORITMOS A
ALGORITMOS B
ALGORITMOS C
SELECCIÓN DE CARACTERÍSTICAS
VALIDACIÓN
AJUSTE DE HIPERPARÁMETROS A, B, C
EVALUACIÓN DE MODELOS A, B, C
PRUEBA
MÉTRICAS FINAL DE MODELO SELECCIONADO

Usamos la función train_test_split de la librería sklearn

```python
import pandas as pd
from sklearn.model_selection import train_test_split

df_train, df_temp = train_test_split(
    df,
    test_size=0.30,
    random_state=42,
    shuffle=True
)

df_val, df_test = train_test_split(
    df_temp,
    test_size=0.50,
    random_state=42,
    shuffle=True
)
```

✓ 0.0s

Tamaño de datos para test

Se descolocan los datos antes de repartirlos

Y ahora el 30% de test lo dividimos en test y validación

## Muestreo estratificado

El muestreo estratificado es el salvavidas de la **representatividad**.

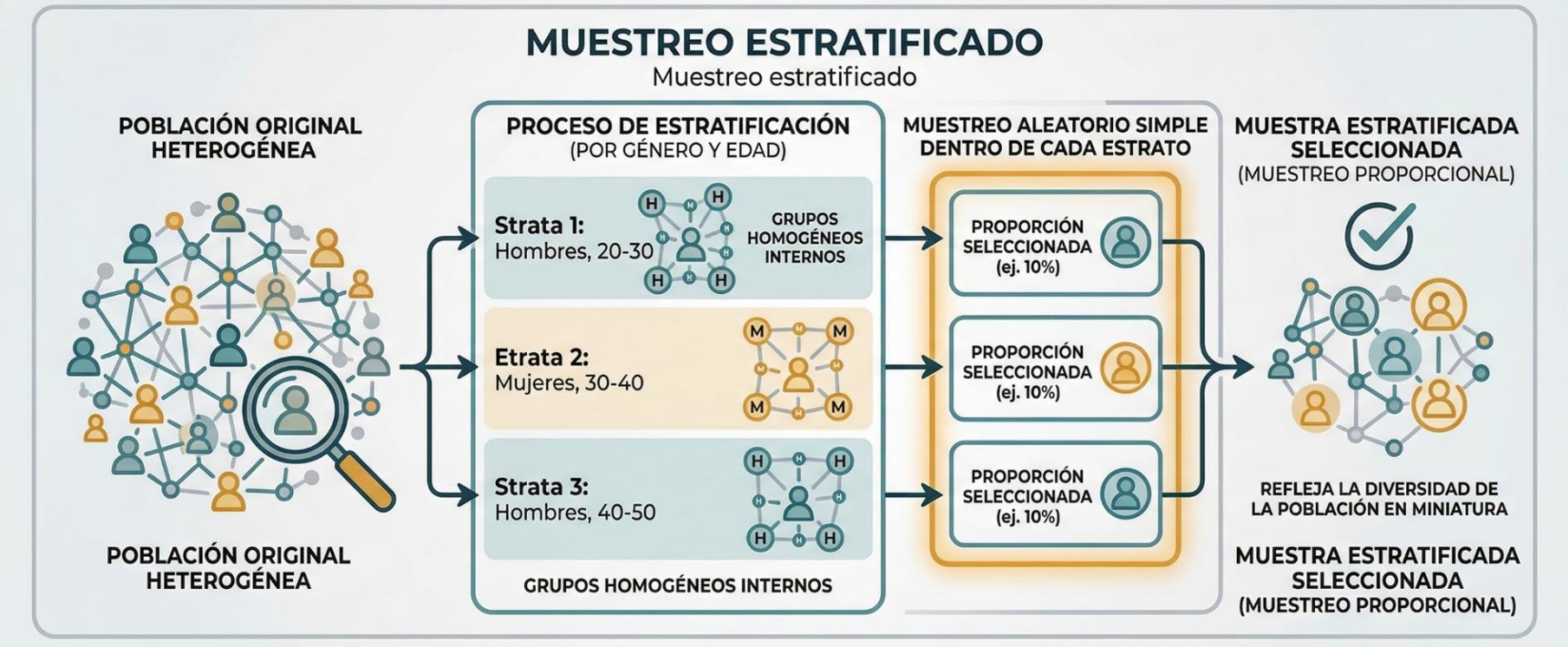
Hay ocasiones en que la aleatoriedad pura puede ser injusta y dejar fuera a grupos críticos, especialmente si son minoritarios.

**Ejemplo**:

Voy a predecir transacciones bancarias fraudulentas y solo hay un 1%. Es muy probable que en el 70% de datos que coja para entrenamiento no haya apenas transacciones fraudulentas.

Solución: divido primero los datos según una característica clave (**estrato**) y luego cojo el 70% de cada estrato.

SAMPLE
MUESTREO
EXPLORE
EXPLORACIÓN
MODIFY
MODIFICACIÓN
MODEL
MODELADO
ASSES
EVALUACIÓN


MUESTREO ESTRATIFICADO
Muestreo estratificado
POBLACIÓN ORIGINAL HETEROGÉNEA
PROCESO DE ESTRATIFICACIÓN
(POR GÉNERO Y EDAD)
Strata 1:
Hombres, 20-30
GRUPOS HOMOGÉNEOS INTERNOS
Etrata 2:
Mujeres, 30-40
Strata 3:
Hombres, 40-50
GRUPOS HOMOGÉNEOS INTERNOS
MUESTREO ALEATORIO SIMPLE DENTRO DE CADA ESTRATO
PROPORCIÓN SELECCIONADA (ej. 10%)
PROPORCIÓN SELECCIONADA (ej. 10%)
PROPORCIÓN SELECCIONADA (ej. 10%)
MUESTRA ESTRATIFICADA SELECCIONADA
(MUESTREO PROPORCIONAL)
REFLEJA LA DIVERSIDAD DE LA POBLACIÓN EN MINIATURA
MUESTRA ESTRATIFICADA SELECCIONADA
(MUESTREO PROPORCIONAL)
POBLACIÓN ORIGINAL HETEROGÉNEA

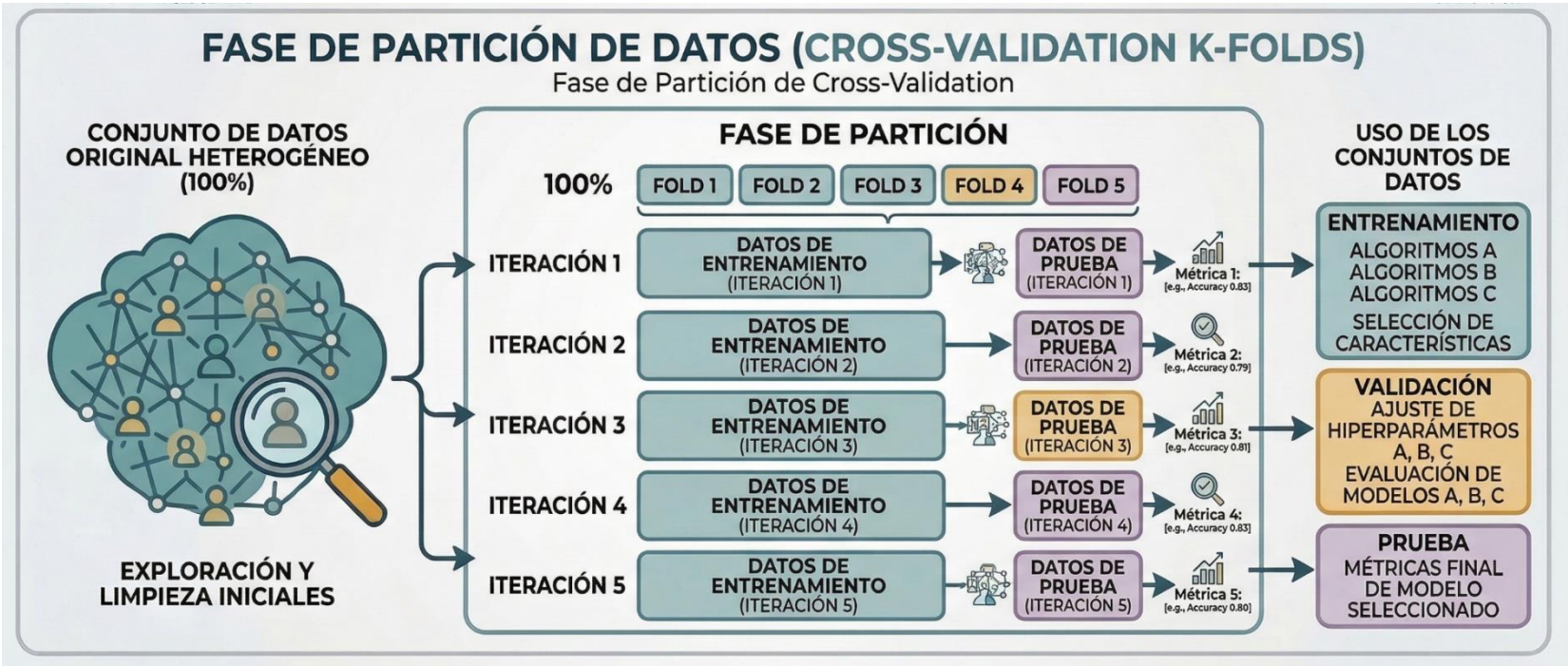
```python
df_train, df_temp = train_test_split(
    df,
    test_size=0.30,
    random_state=42,
    stratify=df['status']
)

df_val, df_test = train_test_split(
    df_temp,
    test_size=0.50,
    random_state=42,
    stratify=df_temp['status']
)
```

Para un reparto estratificado en base al valor de una columna solo hay que indicar el parámetro `stratify`

## Cross Validation (K-fold)

Esta técnica sirve para maximizar el uso de los datos en muestras pequeñas, rotando el segmento de test k veces

SAMPLE
MUESTREO
EXPLORE
EXPLORACIÓN
MODIFY
MODIFICACIÓN
MODEL
MODELADO
ASSES
EVALUACIÓN


FASE DE PARTICIÓN DE DATOS (CROSS-VALIDATION K-FOLDS)
Fase de Partición de Cross-Validation
CONJUNTO DE DATOS ORIGINAL HETEROGÉNEO (100%)
EXPLORACIÓN Y LIMPIEZA INICIALES
FASE DE PARTICIÓN
100%
FOLD 1
FOLD 2
FOLD 3
FOLD 4
FOLD 5
ITERACIÓN 1
DATOS DE ENTRENAMIENTO (ITERACIÓN 1)
DATOS DE PRUEBA (ITERACIÓN 1)
Métrica 1: [e.g., Accuracy 0.83]
ITERACIÓN 2
DATOS DE ENTRENAMIENTO (ITERACIÓN 2)
DATOS DE PRUEBA (ITERACIÓN 2)
Métrica 2: [e.g., Accuracy 0.79]
ITERACIÓN 3
DATOS DE ENTRENAMIENTO (ITERACIÓN 3)
DATOS DE PRUEBA (ITERACIÓN 3)
Métrica 3: [e.g., Accuracy 0.81]
ITERACIÓN 4
DATOS DE ENTRENAMIENTO (ITERACIÓN 4)
DATOS DE PRUEBA (ITERACIÓN 4)
Métrica 4: [e.g., Accuracy 0.83]
ITERACIÓN 5
DATOS DE ENTRENAMIENTO (ITERACIÓN 5)
DATOS DE PRUEBA (ITERACIÓN 5)
Métrica 5: [e.g., Accuracy 0.80]
USO DE LOS CONJUNTOS DE DATOS
ENTRENAMIENTO
ALGORITMOS A
ALGORITMOS B
ALGORITMOS C
SELECCIÓN DE CARACTERÍSTICAS
VALIDACIÓN
AJUSTE DE HIPERPARÁMETROS A, B, C
EVALUACIÓN DE MODELOS A, B, C
PRUEBA
MÉTRICAS FINAL DE MODELO SELECCIONADO

```python
import pandas as pd
from sklearn.model_selection import KFold, StratifiedKFold

k = 5 # Número de folds
skf = StratifiedKFold(n_splits=k, shuffle=True, random_state=42)

for i, (train_index, val_index) in enumerate(skf.split(df_train, df_train['status'])):
    x_train_fold = df_train.iloc[train_index]
    x_val_fold = df_train.iloc[val_index]

    # Aquí es donde se entrenaría el modelo y se evaluaría
```

✓ 0.0s

# 2 LA METODOLOGÍA SEMMA

## 2.3 LA FASE EXPLORE DE SEMMA

# FASE EXPLORE (EXPLORACIÓN)

En la fase de exploración es el paso donde buscas **entender la naturaleza de tus datos** antes de aplicar cualquier algoritmo.

Es la fase en la que se descubren errores, tendencias, valores atípicos y relaciones inesperadas.

## Análisis univariante

Aquí **se analiza cada variable** de forma aislada para entender su distribución.

Algunas cuestiones:

- **Estadística descriptiva**: media, mediana, moda, desviación estándar, mínimos y máximos
- **Visualización**: se usan histogramas para ver si los datos siguen una distribución normal o están sesgados.
- **Identificación de problemas**: valores ausentes, valores atípicos

## Estadística descriptiva

```python
stats_num = ( df[['budget', 'revenue', 'runtime']]
                .describe(
                    percentiles=[.01, .25, .5, .75, .99]
                )
                .style.format("{:,.2f}")
            )
print("Estadísticas de variables numéricas:")
display(stats_num)
```

✓ 0.2s

Voy a analizar estas columnas

Además de las variables estadísticas (media, std, …) también voy a obtener los percentiles.

Con esto le obligo a que no utilice notación científica al mostrar los resultados (más legible)

Estadísticas de variables numéricas:

| | budget | revenue | runtime |
|---|---|---|---|
| count | 9,769.00 | 9,769.00 | 9,769.00 |
| mean | 17,368,525.96 | 55,999,420.34 | 93.36 |
| std | 40,416,635.38 | 163,468,934.89 | 37.95 |
| min | 0.00 | 0.00 | 0.00 |
| 1% | 0.00 | 0.00 | 0.00 |
| 25% | 0.00 | 0.00 | 86.00 |
| 50% | 0.00 | 0.00 | 97.00 |
| 75% | 14,000,000.00 | 24,829,644.00 | 112.00 |
| 99% | 200,000,000.00 | 822,432,432.92 | 177.00 |
| max | 489,900,000.00 | 2,923,706,026.00 | 650.00 |

# Estadística descriptiva

```python
stats_cat = df[['status', 'original_language', 'adult']].describe()
print("\nEstadísticas de variables categóricas:")
display(stats_cat)
```

✓ 0.0s

Ahora analizamos las columnas categóricas

```
Estadísticas de variables categóricas:
```

| | status | original_language | adult |
|---|---|---|---|
| count | 9769 | 9769 | 9769 |
| unique | 6 | 67 | 1 |
| top | Released | en | 0 |
| freq | 9664 | 6670 | 9769 |

Frecuencia del valor más repetido

# Estadística descriptiva

```python
print(f"\nAsimetría del presupuesto (budget): {df['budget'].skew():.2f}")
print(f"Curtosis del presupuesto (budget): {df['budget'].kurt():.2f}")
```

✓ 0.0s

```
Asimetría del presupuesto (budget): 3.62
Curtosis del presupuesto (budget): 16.95
```

También podemos mirar valores como la curtosis o la asimetría

## Visualización de histograma

```python
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")

plt.figure(figsize=(10, 6))

sns.histplot(df['runtime'].dropna(), bins=30, kde=True, color='skyblue')

plt.title('Distribución de la Duración de las Películas (Runtime)', fontsize=15)
plt.xlabel('Duración (minutos)', fontsize=12)
plt.ylabel('Frecuencia (nº de películas)', fontsize=12)

plt.xlim(0, 250)

plt.show()
```

✓ 3.9s

Debemos importar `matplotlib` y `seaborn`

Establecemos el tema (`darkgrid`, `whitegrid`, `dark`, `white`, `ticks`)

Creamos el lienzo donde vamos a representar el histograma. Se pasa el tamaño (ancho, alto)

## Visualización de histograma

```
import ma
import se

sns.set_theme(style="whitegrid")

plt.figure(figsize=(10, 6))

sns.histplot(df['runtime'].dropna(), bins=30, kde=True, color='skyblue')

plt.title('Distribución de la Duración de las Películas (Runtime)', fontsize=15)
plt.xlabel('Duración (minutos)', fontsize=12)
plt.ylabel('Frecuencia (nº de películas)', fontsize=12)

plt.xlim(0, 250)

plt.show()
```

✓ 3.9s

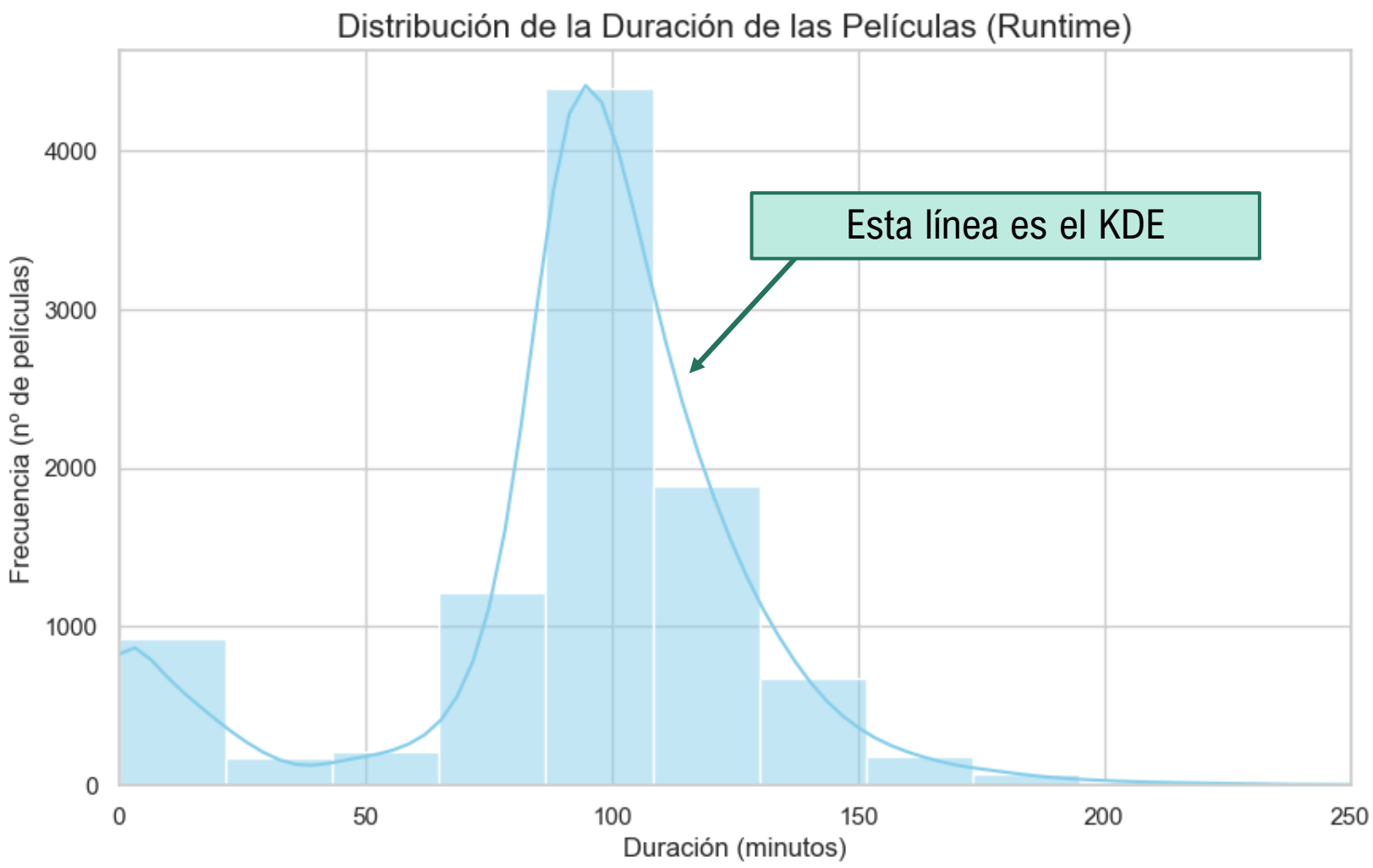
Eliminamos valores nulos antes de representarlos

El histograma tendrá 30 barras

kde es la **Estimación de Densidad del Núcleo**, es una línea que suaviza los datos para mostrar la forma de la distribución independientemente del número de bins escogidos

**Color de las barras**. Pueden ser nombres estándar (`red`, `navy`, `darkgreen`, `firebrick`), códigos hexadecimales (`#4B0002`) o abreviaturas de Matplitlib (`r`, `g`, `b`, `k`)

# Visualización de histograma



# Valores ausentes

```python
nulos_por_columna = df.isnull().sum()
print("** Nulos por columna **")
print(nulos_por_columna)
```

✓ 0.0s

```
** Nulos por columna **
id                     0
title                  0
original_title         0
overview             160
release_date          61
runtime                1
budget                 1
revenue                1
vote_average           1
vote_count             1
popularity             1
poster_path          338
backdrop_path       1519
status                 1
tagline             3803
homepage            6337
original_language      1
adult                  1
```

En cada columna, suma los valores, teniendo en cuenta que interpreta `True` como `1` y `False` como `0`

La función `isnull()` reemplazar el valor de cada columna por True/False sea nulo o no

```python
df.isnull()
```

✓ 0.0s

| | id | title | original_title | overview | release_date | runtime | budget | rev |
|---|---|---|---|---|---|---|---|---|
| 0 | False | False | False | False | False | False | False | |
| 1 | False | False | False | False | False | False | False | |
| 2 | False | False | False | False | False | False | False | |
| 3 | False | False | False | False | False | False | False | |
| 4 | False | False | False | False | False | False | False | |
| ... | ... | ... | ... | ... | ... | ... | ... | |

## Valores ausentes

```python
porcentaje_nulos = (df.isnull().sum() / len(df)) * 100
print("** Porcentaje de nulos **")
print(porcentaje_nulos)
```

✓ 0.0s

```
** Porcentaje de nulos **
id                    0.000000
title                 0.000000
original_title        0.000000
overview              1.637499
release_date          0.624296
runtime               0.010234
budget                0.010234
revenue               0.010234
vote_average          0.010234
vote_count            0.010234
popularity            0.010234
poster_path           3.459216
backdrop_path        15.546003
status                0.010234
tagline              38.921298
homepage             64.855184
original_language     0.010234
adult                 0.010234
video                 0.010234
```

Mejor que un valor absoluto, es preferible conocer el porcentaje de nulos de cada columna

## Análisis multivariante

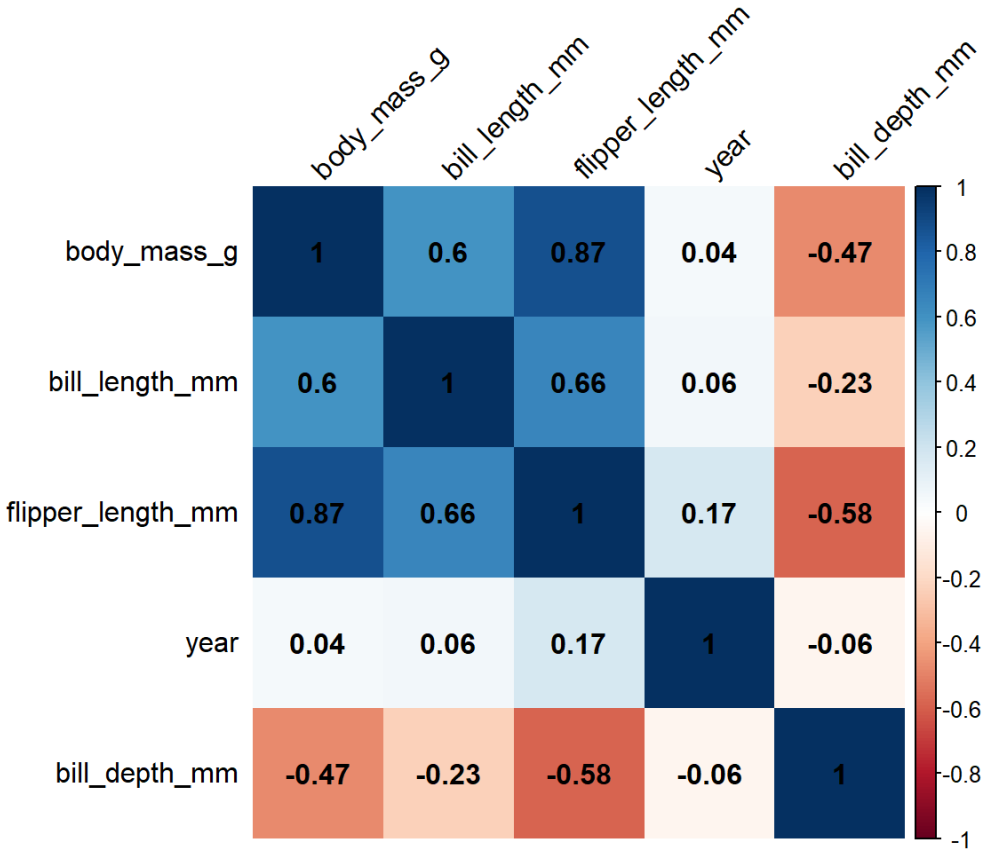
Aquí se busca entender cómo interactúan las variables entre sí, especialmente con la variable objetivo.

Algunos instrumentos de esta fase:

- **Matrices de correlación**
- **Gráficos de dispersión**

## Matrices de correlación

Es un mapa de calor que dice qué tan relacionadas están las variables de tu *dataset* entre sí.



Las filas y las columnas tienen las mismas variables

**Correlación positiva** (entre 0 y +1): cuando una variable sube la otra también

**Correlación negativa** (entre -1 y 0): cuando una variable sube la otra baja

| | body_mass_g | bill_length_mm | flipper_length_mm | year | bill_depth_mm |
|---|---|---|---|---|---|
| body_mass_g | 1 | 0.6 | 0.87 | 0.04 | -0.47 |
| bill_length_mm | 0.6 | 1 | 0.66 | 0.06 | -0.23 |
| ipper_length_mm | 0.87 | 0.66 | 1 | 0.17 | -0.58 |
| year | 0.04 | 0.06 | 0.17 | 1 | -0.06 |
| bill_depth_mm | -0.47 | -0.23 | -0.58 | -0.06 | 1 |

Un valor próximo a 0 indica que las variables no tienen ninguna relación

La diagonal indica la relación entre una variable consigo mismo, por lo que siempre es 1

## ¿Para qué sirve?

- **Identificar predictores**: hay que buscar variables que tengan una correlación alta con la variable objetivo
- **Detectar multicolinealidad (redundancia)**: si dos variables tienen una correlación muy alta (p.e. 0.95) significa que aportan casi la misma información, por lo que se podría descartar una.
- **Descartar ruido**: si una variable tiene correlación cercana a 0 con todas las demás y con el objetivo, probablemente sea *basura* estadística que no aporta nada al algoritmo.

Me quedo solo con las columnas numéricas

```python
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

df_numeric = df.select_dtypes(include=[np.number]).drop(columns=['id'], errors='ignore')

corr_matrix = df_numeric.corr()

plt.figure(figsize=(12, 8))
sns.heatmap(
    corr_matrix,
    annot=True,
    fmt=".2f",
    cmap='RdBu_r',
    center=0,
    linewidths=0.5
)

plt.title('Matriz de Correlación de
plt.show()
```

✓ 1.4s

Y me acuerdo de quitar el id, que, aunque es un número, es un identificador

La función `corr()` me genera la matriz de correlación

```python
corr_matrix
```

✓ 0.0s

| | runtime | budget | revenue |
|---|---|---|---|
| runtime | 1.000000 | 0.288813 | 0.250214 |
| budget | 0.288813 | 1.000000 | 0.752135 |
| revenue | 0.250214 | 0.752135 | 1.000000 |

```
impor
impor
impor

df_nu                                                    drop(columns=['id'], errors='ignore')

corr_matrix = df_numeric.corr()

plt.figure(figsize=(12, 8))
sns.heatmap(
    corr_matrix,
    annot=True,
    fmt=".2f",
    cmap='RdBu_r',
    center=0,
    linewidths=0.5
)

plt.title('Matriz de Correlación
plt.show()
```

✓ 1.4s

Ahora vamos a representarla gráficamente con un mapa de calor. Comenzamos inicializando la figura

`heatmap()` es la función de Seaborn que nos sirve para crear mapas de calor

`annot=True` muestra los valores numéricos en cada celda

Limitamos a 2 decimales para facilitar la lectura

```python
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

df_numeric = df.select_dtypes(include=[np.number]).drop(columns=['id'], errors='ignore')

corr_matrix = df_numeric.corr()

plt.figure(figsize=(12, 8))
sns.heatmap(
    corr_matrix,
    annot=True,
    fmt=".2f",
    cmap='RdBu_r',
    center=0,
    linewidths=0.5
)

plt.title('Matriz de Correlac
plt.show()
```

✓ 1.4s

Colores mostrados en el mapa de calor.
Algunos valores son:

- **RdBu**_r: escala de rojo (positivo) a azul (negativo)
- **coolwarm**: el más común, también rojo (positivo) a azul (negativo)
- **PRGn**: púrpura y verde
- **seismic**: azul y rojo muy intensos, con transiciones muy rápidas.
- **viridis**: mapa por defecto de Matplotlib. De morado a amarillo
- **magma**: de negro y morado a naranja muy brillane

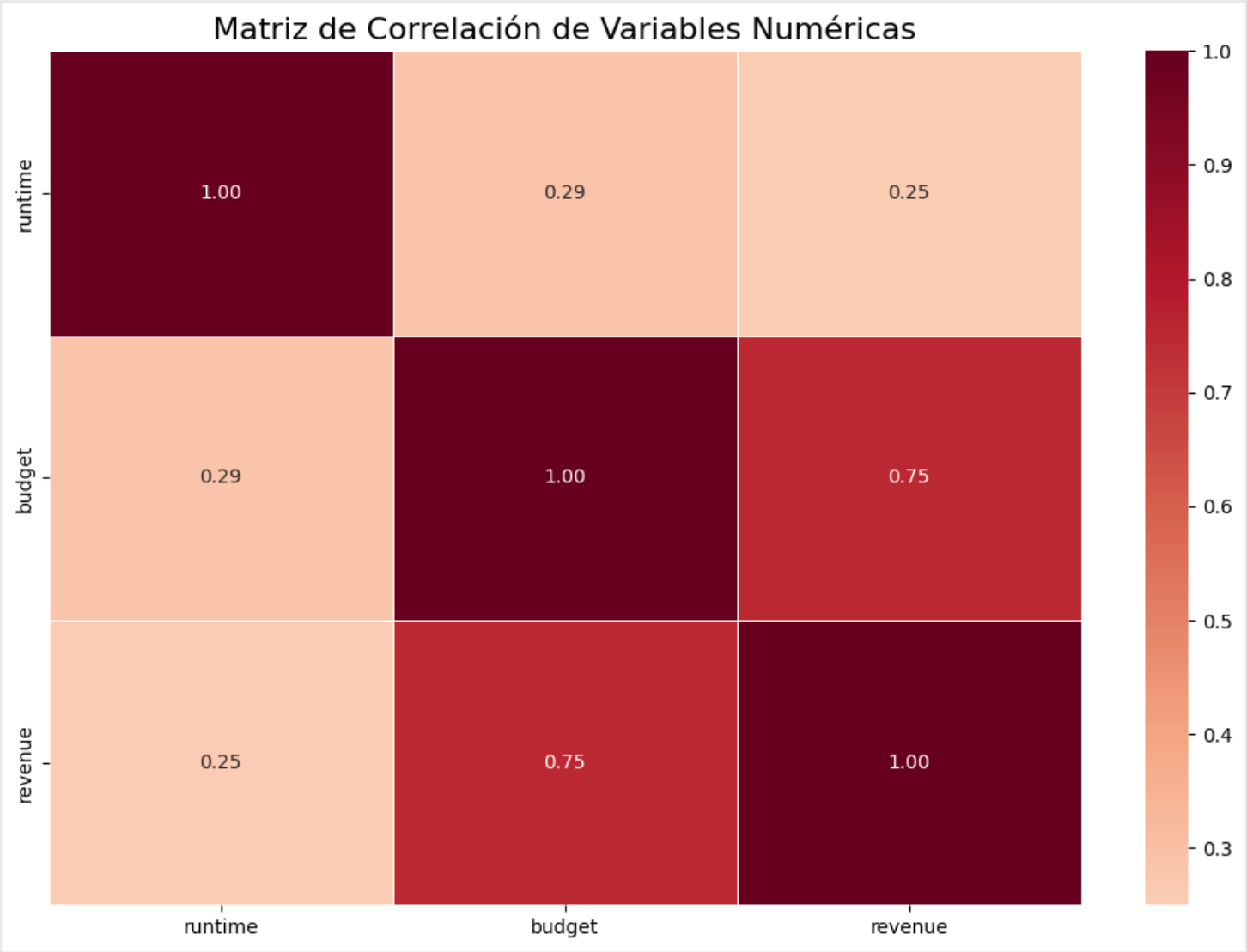
```python
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

df_numeric = df.select_dtypes(include=[np.number]).drop(columns=['id'], errors='ignore')

corr_matrix = df_numeric.corr()

plt.figure(figsize=(12, 8))
sns.heatmap(
    corr_matrix,
    annot=True,
    fmt=".2f",
    cmap='RdBu_r',
    center=0,
    linewidths=0.5
)

plt.title('Matriz de Correlación de Variables Numéricas', fontsize=16)
plt.show()
```

✓ 1.4s

El color neutro se fija en el 0

Añadimos una pequeña separación entre celdas

Mostramos la figura

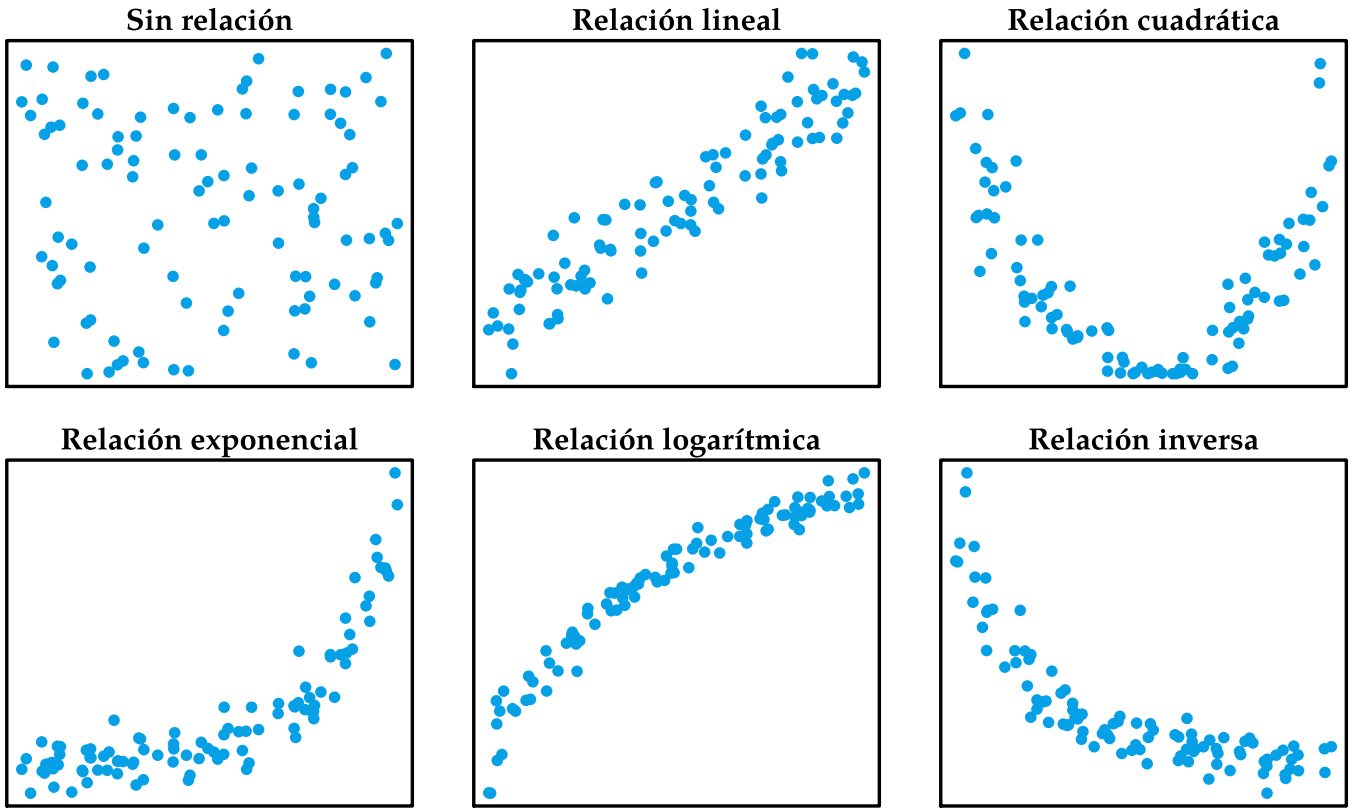

Matriz de Correlación de Variables Numéricas
runtime
budget
revenue
1.00
0.29
0.25
0.29
1.00
0.75
0.25
0.75
1.00
runtime
budget
revenue
1.0
0.9
0.8
0.7
0.6
0.5
0.4
0.3

## Gráficos de dispersión

Es un diagrama matemático que utiliza coordenadas cartesianas (x, y) para mostrar los valores de dos variables de un conjunto de datos.

Cada punto en el gráfico representa una observación individual.

SAMPLE
MUESTREO
EXPLORE
EXPLORACIÓN
MODIFY
MODIFICACIÓN
MODEL
MODELADO
ASSES
EVALUACIÓN


Sin relación
Relación lineal
Relación cuadrática
Relación exponencial
Relación logarítmica
Relación inversa

¿Para qué sirve?

- **Dirección de la correlación**: si los puntos suben de izquierda a derecha hay una correlación positiva, si bajan es negativa.
- **Fuerza de la relación**: si los puntos están muy pegados formando casi una línea la relación es fuerte, si están dispersos es débil.
- **Forma de la relación**: puede ser lineal (los puntos siguen una línea recta) o no lineal (forman una curva). Esto es importante porque la matriz de correlación no refleja las relaciones no lineales.
- **Detección de valores atípicos**: permite ver visualmente si hay valores atípicos.

```
import seaborn as sns
import matplotlib.pyplot
import pandas as pd

cols_interes = ['budget', 'revenue', 'vote_average']
for col in cols_interes:
    df[col] = pd.to_numeric(df[col], errors='coerce')

df_plot = df.dropna(subset=cols_interes)

df_plot = df_plot[(df_plot['budget'] >

pl
sn

    palette='viridis',
    alpha=0.5
)

plt.title('Relación Presupuesto vs Ingr
plt.show()
```

Me aseguro de que las columnas tengan un valor numérico

La función **`dropna()`** elimina las filas que tienen valores nulos.

El parámetro **`subset`** sirve para que elimine las filas solo si el valor nulo está en una de esas columnas.

Otros parámetros de `dropna()` son:

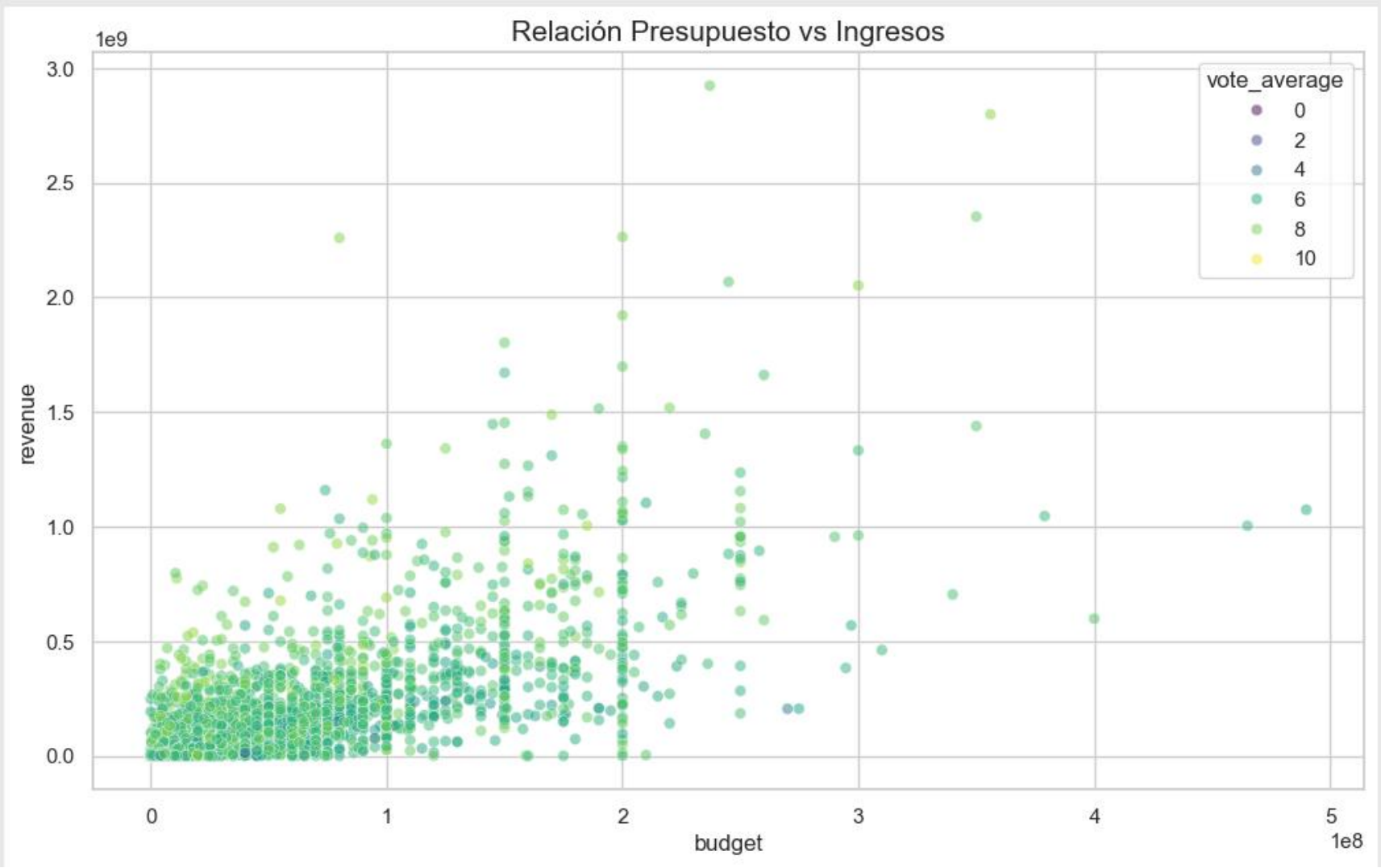
- `how='all'`: eliminar filas que tengan todo nulo.
- `axis=1`: por defecto, elimina la columna entera si tiene un único valor nulo, por lo que se suele combinar con `thresh`.
- `thresh=num`: en combinación con `axis=1`, solo elimina columnas que tengan más de *num* nulos.

```python
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

cols_interes = ['budget', 'reve
for col in cols_interes:
    df[col] = pd.to_numeric(df[col], errors='coerce')

df_plot = df.dropna(subset=cols_interes)

df_plot = df_plot[(df_plot['budget'] > 0) & (df_plot['revenue'] > 0)]

plt.figure(figsize=(12, 7))
sns.scatterplot(
    data=df_plot,
    x='budget',
    y='revenue',
    hue='vote_average',
    palette='viridis',
    alpha=0.5
)

plt.title('Relación Presupuest                                    fontsize=15)
plt.show()
```

Filtro las filas que pudiera haber con presupuesto o ingresos con valor 0 y que pueden distorsionar la gráfica

Uso la función `scatterplot()` de Seaborn para dibujar la gráfica.

Indico qué variable se va a representar en cada eje. La tercera variable se indica con `hue` y se representa con el color


Relación Presupuesto vs Ingresos
1e9
3.0
2.5
2.0
1.5
1.0
0.5
0.0
revenue
0
1
2
3
4
5
1e8
budget
vote_average
0
2
4
6
8
10

# 2 LA METODOLOGÍA SEMMA

## 2.4 LA FASE MODIFY DE SEMMA

# FASE MODIFY (MODIFICACIÓN)

Aquí es donde se aplican las **técnicas de calidad** sobre los datos a partir de la información obtenida en la fase de exploración.

Consiste en aplicar operaciones matemáticas o lógicas para que los datos sean más digeribles para el algoritmo:

- Tratamiento de nulos
- Transformación de variables
- Escalado y normalización (Z-score)
- Feature selection

## Tratamiento de nulos

En la fase de exploración identificamos los valores nulos.

En esta fase es cuando debemos decidir **qué hacer con los valores nulos**.

Las posibilidades son:

- Eliminar la fila o columna completa
- Imputación simple
- Imputación por valor constante

## Eliminación de la fila

Es la política más conservadora pero también la más cara en términos de pérdida de información.

Conviene hacerlo cuando **la fila tiene nulos en la variable objetivo** (la que queremos predecir) o cuando **faltan datos en muchas columnas** de esa fila.

El **riesgo** es que, si el dataset es muy pequeño, podemos quedarnos sin datos suficientes.

## Eliminación de la columna

Cuando una característica tiene un porcentaje de nulos muy alto.

```python
df.isnull().sum()
```

✓ 0.0s

Con **`isnull()`** puedo obtener un resumen del número de nulos que hay en cada columna

```
id                     0
title                  0
original_title         0
overview             160
release_date          61
runtime                1
budget                 1
```

```python
df.dropna(subset=['budget'])
```

✓ 0.0s

La función **`dropna()`** con el parámetro **`subset`** sirve para eliminar todas las filas en las que el valor de la columna indicada en `subset` sea nulo

| | id | title | original_ti | | | | dget | reve |
|---|---|---|---|---|---|---|---|---|
| 0 | 2 | Ariel | Ariel | A Finnish man goes to the city to find a job a... | 1988-10-21 | 73.0 | 0.0 | |
| 1 | 11 | Star Wars | Star Wars | Princess Leia is captured and held | 1977-05-25 | 121.0 | 11000000.0 | 7753980 |

```python
df.dropna(subset=['budget', 'revenue'], how='all')
```

✓ 0.0s

| | id | title | original_title | | | | |
|---|---|---|---|---|---|---|---|
| 0 | 2 | Ariel | Ariel | | | | |
| 1 | 11 | Star Wars | Star Wars | 1977-05-25 | 121.0 | 11000000.0 | 7753980 |

Si en **`subset`** incluyo varias columnas, deberé combinarlo con el parámetro **`how`**, cuyos valores pueden ser:

- **`all`**: elimina la fila si los valores de todas las columnas son nulos
- **`any`**: elimina la fila si por lo menos un valor de una columna es nulo

```python
df.dropna(how='any')
```

✓ 0.0s

| | id | title | original_title | get | reve |
|---|---|---|---|---|---|
| 1 | 11 | Star Wars | Star Wars | 0.0 | 7753980 |

Si usamos **`how`** solo, podemos borrar todas las filas que tengan por lo menos un valor nulo (`any`) o que tengan todos los valores nulos (`all`)

```
df.dropna(thresh=5)
```

✓ 0.0s

| | id | title | original_title | overview | release_date | runtime | budget | rever |
|---|---|---|---|---|---|---|---|---|
| 1 | 11 | Star Wars | Sta | Princess Leia is captured and | | | | 77539800 |
| 2 | 12 | Finding Nemo | Finding Nemo | adventurous young | 2003-05-30 | 100.0 | 94000000.0 | 94033553 |

Por último, está la posibilidad de establecer un valor umbral con **`thresh`**.
En este ejemplo se eliminarán todas las filas que no tengan 5 o más valores no nulos.

## Imputación simple

Consiste en reemplazar los nulos por un valor estadístico:

- **Media**: útil en variables numéricas con una distribución normal y sin outliers extremos (ya que la media es muy sensible a estos valores)
- **Mediana**: en variables con outliers o distribuciones sesgadas puede ser la opción más segura.
- **Moda**: es el valor que se suele utilizar en variables categóricas.

## Imputación por valor constante

Se puede hacer cuando nulo tiene un significado propio.

Por ejemplo, en un dataset de películas, si num_premios tiene un valor nulo es muy probable que esa película no tenga premios.

```python
media = df['budget'].mean()
mediana = df['budget'].median()
moda = df['budget'].mode()[0]

df['budget'].fillna(media)
```

✓ 0.0s

Las funciones para obtener la media, la mediana y la moda son `mean()`, `median()` y `mode()` respectivamente

La función `mode()` no devuelve un único valor, sino una lista de valores, por lo que nos quedamos con el primero.

Con **`fillna()`** rellenamos todos los campos cuyo valor sea nulo con lo que le pasemos como parámetro.

## Normalización y estandarización

Muchos algoritmos son sensibles a las escalas.

Dos métodos utilizados son:

- **Min-Max Scaling (normalización):** escala los valores a un rango fijo, usualmente [0, 1].

$$X_{scaled} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

- **Z-score (estandarización):** transforma los datos para que tengan una media de 0 y una desviación estándar de 1. Ideal cuando los datos siguen una distribución normal.

Score

Mean

$$Z = \frac{x - \mu}{\sigma}$$

SD

Para la **normalización** (escalado min-max) podemos usar MinMaxScaler de sklearn

```python
from sklearn.preprocessing import MinMaxScaler
import pandas as pd

df['popularity'] = pd.to_numeric(df['popularity'], errors='coerce')

mediana_pop = df['popularity'].median()
df['popularity'] = df['popularity'].fillna(mediana_pop)

scaler = MinMaxScaler()
df['popularity_minmax'] = scaler.fit_transform(df[['popularity']])

print("Conversión y escalado completados.")
print(df[['popularity', 'popularity_minmax']].head())
```

✓ 0.0s

En este ejemplo, en lugar de eliminar los nulos, los reemplazo por la mediana.

```
Conversión y escalado completados.
   popularity  popularity_minmax
0      5.0295           0.010587
1     23.1666           0.060434
2     15.7935           0.040170
3     22.7297           0.059233
4      8.6764           0.020610
```

Para aplicar el **Z-Score** tenemos la función StandardScaler

```python
from sklearn.preprocessing import StandardScaler
import pandas as pd

df['vote_average'] = pd.to_numeric(df['vote_average'], errors='coerce')
df['vote_average'] = df['vote_average'].fillna(df['vote_average'].mean())

scaler_z = StandardScaler()

df['vote_avg_zscore'] = scaler_z.fit_transform(df[['vote_average']])

print(f"Media tras Z-score (cercana a 0): {df['vote_avg_zscore'].mean():.2f}")
print(f"Desviación estándar tras Z-score: {df['vote_avg_zscore'].std():.2f}")

print(df[['vote_average', 'vote_avg_zscore']].head())
```

✓ 0.0s

En este caso, reemplazo los nulos por la media

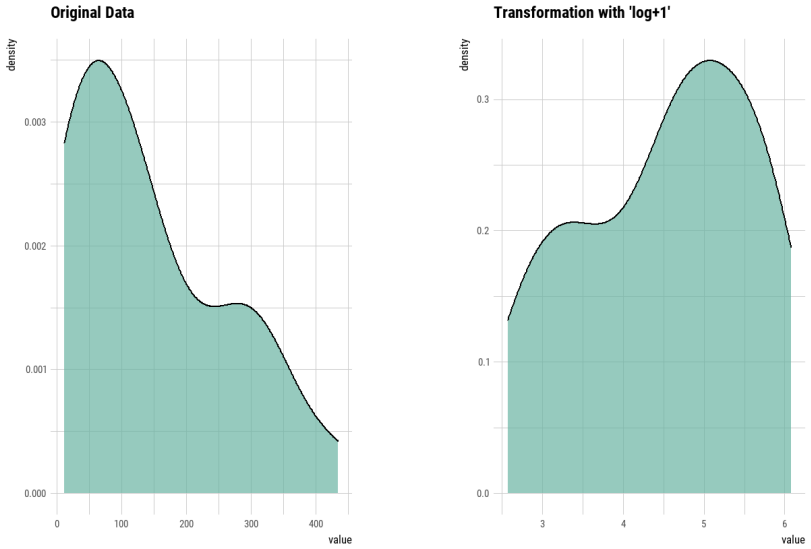
```
Media tras Z-score (cercana a 0): -0.00
Desviación estándar tras Z-score: 1.00
   vote_average  vote_avg_zscore
0         7.102         0.706934
1         8.202         1.180938
2         7.817         1.015037
3         8.462         1.292976
4         8.002         1.094756
```

## Transformaciones para corregir el sesgo

A veces los datos están amontonados en los valores bajos y tienen una cola muy larga en los altos, lo que confunde a los modelos lineales.

Para solucionarlo:

- **Transformación logarítmica**: aplicar log(x) ayuda a comprimir los valores muy grandes y expandir los pequeños, aproximando más la distribución a una campana de Gauss

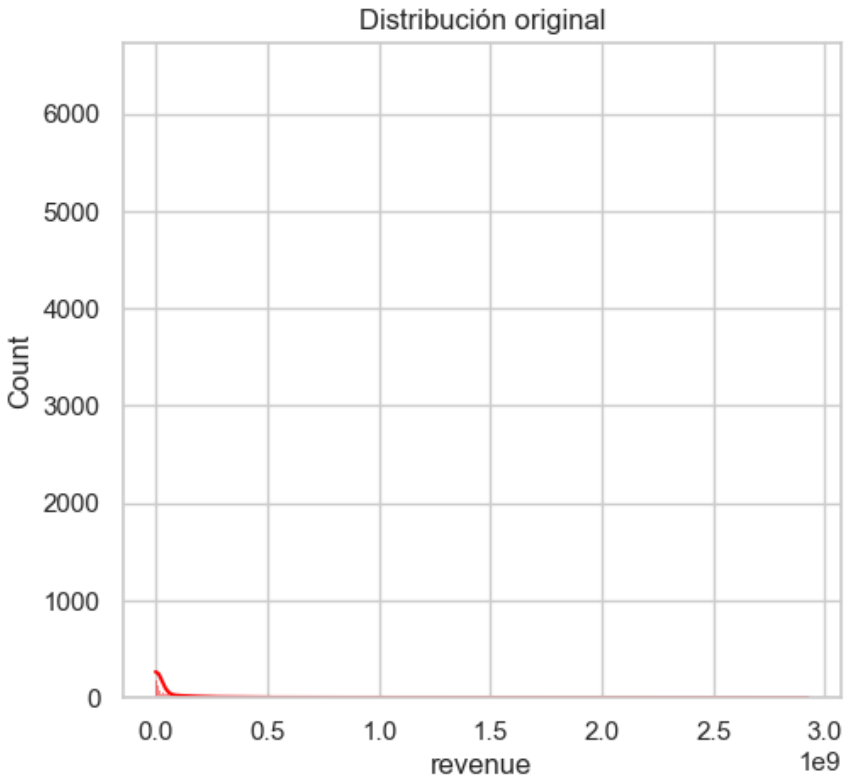


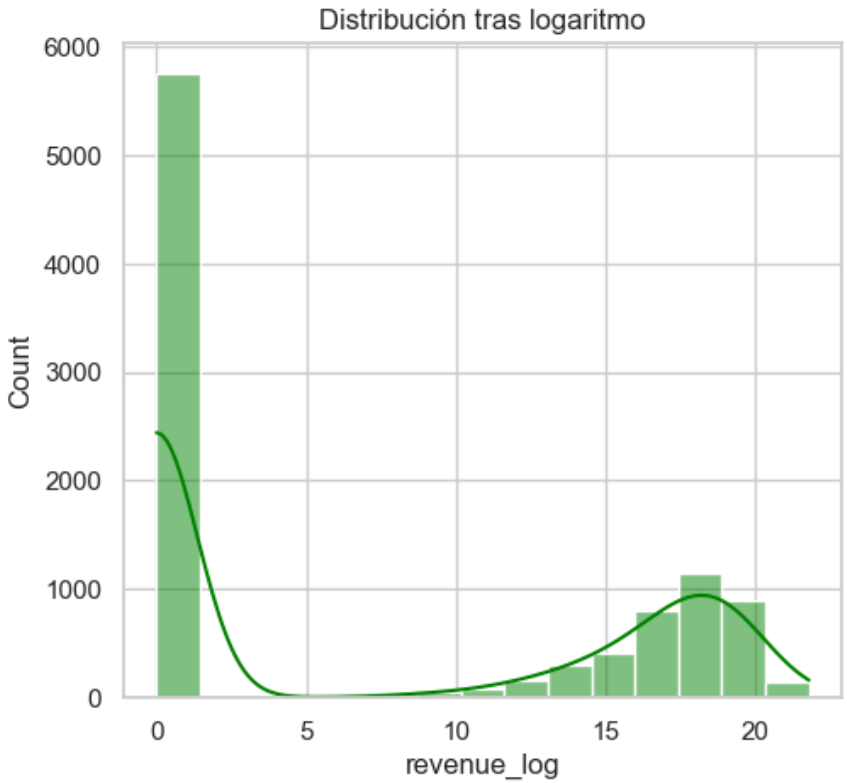
La función **`np.log1p(x)`** calcula log(1+x) para evitar problemas si hay algún valor 0, ya que log(0) = infinito

```python
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df['revenue_log'] = np.log1p(df['revenue'])

print(f"Sesgo original de Revenue: {df['revenue'].skew():.2f}")
print(f"Sesgo tras transformación log: {df['revenue_log'].skew():.2f}")

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
sns.histplot(df['revenue'], kde=True, color='red')
plt.title('Distribución original')

plt.subplot(1, 2, 2)
sns.histplot(df['revenue_log'], kde=True, color='green')
plt.title('Distribución tras logaritmo')

plt.show()
```

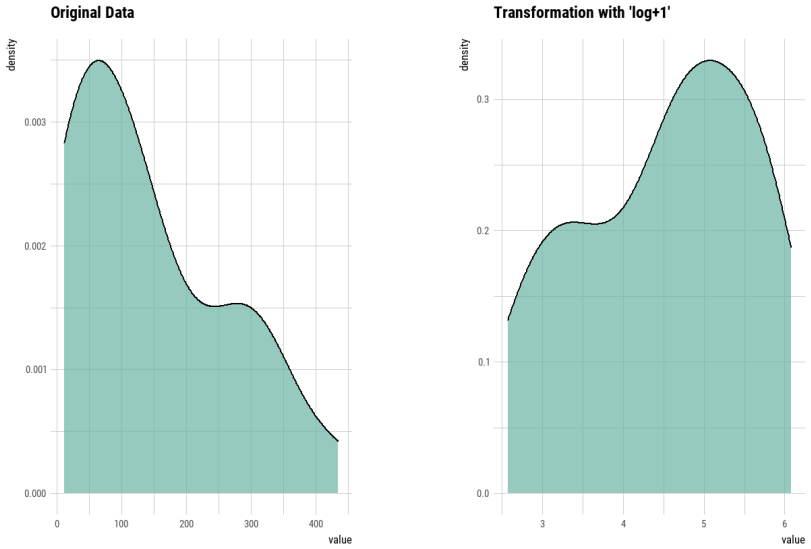
Usamos `subplot()` para indicar que vamos a dividir la figura en dos diagramas

```
Sesgo original de Revenue: 6.00
Sesgo tras transformación log: 0.45
```


Distribución original
Count
6000
5000
4000
3000
2000
1000
0
0.0
0.5
1.0
1.5
2.0
2.5
3.0
1e9
revenue


Distribución tras logaritmo
Count
6000
5000
4000
3000
2000
1000
0
0
5
10
15
20
revenue_log

- **Raíz cuadrada**: similar al logaritmo, pero menos agresiva. Se usa mucho en datos de conteo.

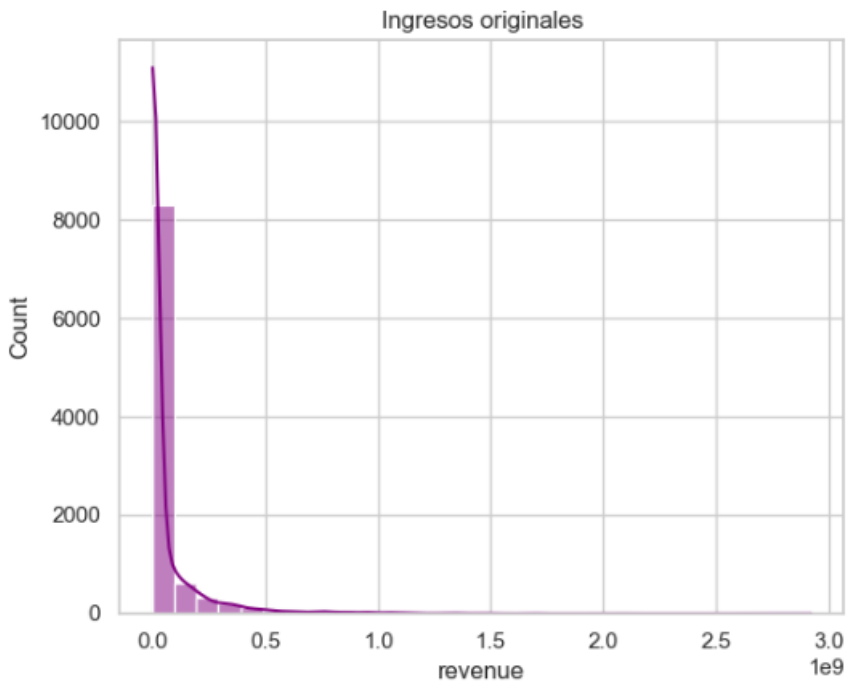


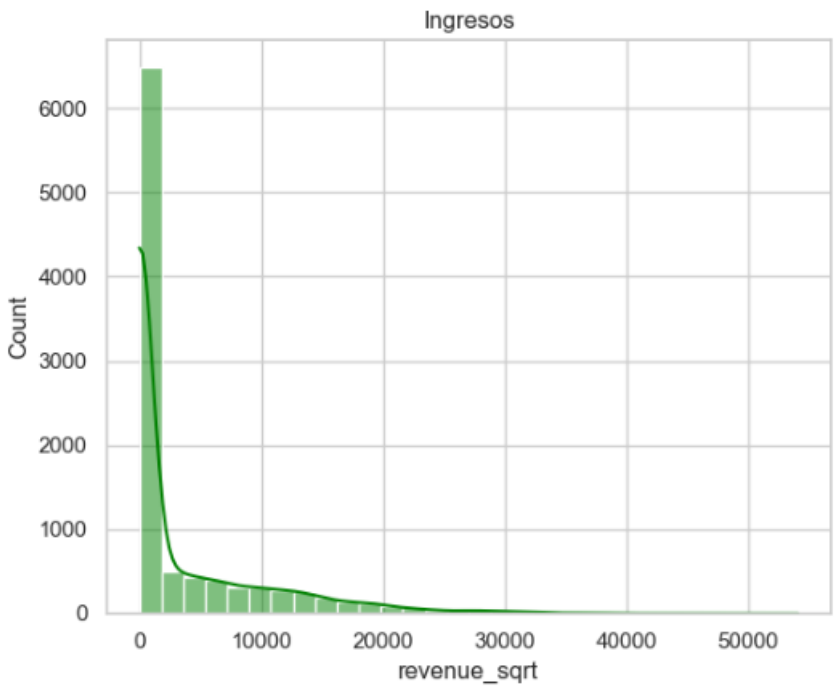
En este caso uso la función `np.sqrt()` de Numpy

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df['revenue'] = pd.to_numeric(df['revenue'], errors='coerce')
df['revenue'] = df['revenue'].fillna(0)

df['revenue_sqrt'] = np.sqrt(df['revenue'])

print(f"Sesgo original de ingresos: {df['revenue'].skew():.2f}")
print(f"Sesgo tras raíz cuadrada: {df['revenue_sqrt'].skew():.2f}")

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
sns.histplot(df['revenue'], kde=True, color='purple', bins=30)
plt.title('Ingresos originales')

plt.subplot(1, 2, 2)
sns.histplot(df['revenue_sqrt'], kde=True, color='green', bins=30)
plt.title('Ingresos')

plt.tight_layout()
plt.show()
```

```
Sesgo original de ingresos: 6.00
Sesgo tras raíz cuadrada: 2.32
```

Observa cómo el sesgo ha descendido, pero mucho menos.


Ingresos originales
Count
10000
8000
6000
4000
2000
0
0.0
0.5
1.0
1.5
2.0
2.5
3.0
1e9
revenue


Ingresos
Count
6000
5000
4000
3000
2000
1000
0
0
10000
20000
30000
40000
50000
revenue_sqrt

## Discretización o binning (agrupamiento)

Consiste en **convertir una variable numérica en categórica**.

Sirve para **reducir el ruido**. P.e. en lugar de que el modelo intente aprender la diferencia exacta entre 31 y 32 años, se crean rangos de edad: joven (18-30), adulto (31-50), …

Los modelos de IA (p.e. árboles de decisión) funcionan mejor con categorías que con números infinitos

Dos métodos:

- **Igual ancho**: divisiones de igual tamaño
- **Igual frecuencia**: cada grupo tiene el mismo número de registros.

# Discretización de igual ancho

```python
import pandas
import numpy as np

df['runtime'] = pd.to_numeric(df['runtime'], errors='coerce')
df['runtime'] = df['runtime'].fillna(df['runtime'].median())

cortes = [0, 60, 120, 200, np.inf]
nombres = ['Corta', 'Media', 'Larga', 'Épica']

df['runtime_cat'] = pd.cut(df['runtime'], bins=cortes, labels=nombres)

print("Conteo de películas por categoría de duración:")
print(df['runtime_cat'].value_counts())

print("\nEjemplo de los datos transformados:")
print(df[['runtime', 'runtime_cat']].head(5))
```

✓ 0.0s

Indicamos los valores que entran en cada categoría. Utilizo **`np.inf`** para indicar que el último rango es desde 200 minutos hasta lo que sea

Uso la función **`pd.cut()`**, a la que hay que indicarle los puntos de corte y el nombre que corresponde a cada uno.

```
Conteo de películas por categoría de duración:
runtime_cat
Media    6959
Larga    1512
Corta     822
Épica      39
Name: count, dtype: int64

Ejemplo de los datos transformados:
   runtime runtime_cat
0     73.0       Media
```

# Discretización de igual frecuencia

```python
import pandas
import numpy as np

df['popularity'] = pd.to_numeric(df['popularity'], errors='coerce')
df['popularity'] = df['popularity'].fillna(df['popularity'].median())

df['pop_level'] = pd.qcut(
    df['popularity'],
    q=4,
    labels=['Baja', 'Media', 'Alta', 'Muy Alta']
)

print("Conteo de películas por nivel de popularidad:")
print(df['pop_level'].value_counts())

intervalos = pd.qcut(df['popularity'], q=4).unique()
print(f"\nIntervalos calculados:\n{intervalos}")
```

✓ 0.0s

En este caso uso la función **`pd.qcut()`**, a la que hay que pasarle el número de divisiones y el nombre de cada una

```
Conteo de películas por nivel de popularidad:
pop_level
Media       2444
Baja        2443
Muy Alta    2443
Alta        2441
Name: count, dtype: int64

Intervalos calculados:
[(4.995, 6.349], (6.349, 365.031], (4.36, 4.995], (1.1760000000000002, 4.36]]
Categories (4, interval[float64, right]): [(1.1760000000000002, 4.36] < (4.36, 4.995] < (4.995, 6
```

## Creación de variables (feature engineering)

A partir de variables existentes, se generan nuevas que aportan más valor predictivo.

Ejemplos:

- **Ratios o proporciones**: en lugar de mirar "gasto total" o "límite de crédito" por separado, se crea la variable "Porcentaje de uso de tarjeta"
- **Extracción de fechas**: convertir una fecha en variables categóricas como "día de la semana", "es fin de semana" o "mes"

## Codificación de variables categóricas

Las máquinas no entienden palabras, solo números.

Algunos métodos:

- **One-Hot encoding**: crea una columna nueva por cada categoría y asigna valores 0 o 1

| Pet type | Pet Code |
|---|---|
| cats | 0 |
| dogs | 1 |
| birds | 2 |

categorical data

| Pet type | is cat | is dog | is bird |
|---|---|---|---|
| cats | 1 | 0 | 0 |
| dogs | 0 | 1 | 0 |
| birds | 0 | 0 | 1 |

one hot encoding

- **Ordinal encoding**: si las categorías tienen un orden lógico (p.e. bajo, medio, alto) se convierten en números consecutivos.

| Original Encoding | Ordinal Encoding |
|---|---|
| Poor | 1 |
| Good | 2 |
| Very Good | 3 |
| Excellent | 4 |

- **Target encoding**: consiste en sustituir cada categoría por la media de la variable objetivo para esa categoría específica.

| Ciudad | Fraude (Target) |
|---|---|
| Madrid | 0 |
| Madrid | 1 |
| Madrid | 0 |
| Barcelona | 1 |
| Barcelona | 1 |

De 3 transacciones en Madrid, 1 fue fraude.
Media = 1/3 = **0.33**

Las 2 transacciones en Barcelona fueron fraude.
Media = 2/2 = **1.0**

**Ventajas**:

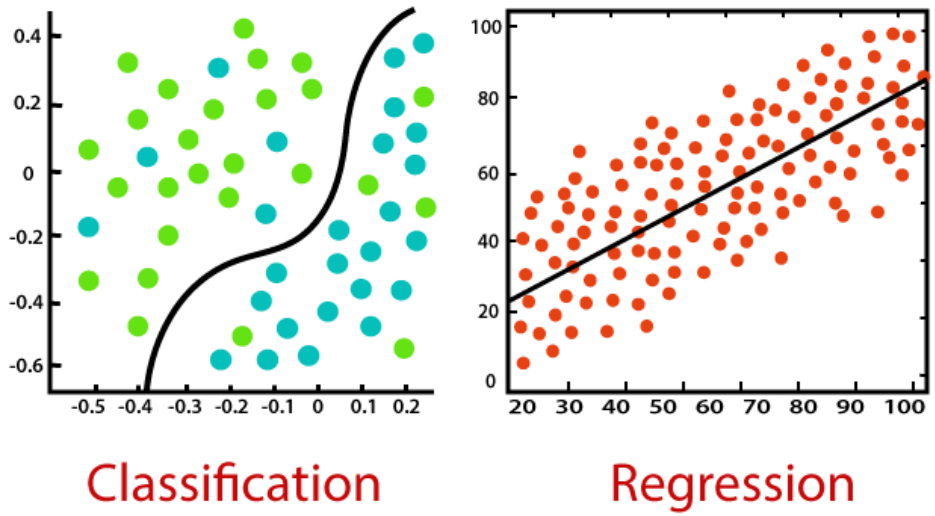
- Solo añade una columna
- Captura la relación directa ya que le está diciendo al modelo qué categorías son más peligrosas.
- Ideal para categorías de alta cardinalidad

# 2 LA METODOLOGÍA SEMMA

## 2.5 LA FASE MODEL DE SEMMA

# FASE MODEL (MODELADO)

En esta fase es donde **se selecciona la técnica adecuada** según la naturaleza del problema (regresión vs clasificación)



Esta fase se sale del ámbito de esta asignatura, por lo que no profundizaremos más en ella.

# 2 LA METODOLOGÍA SEMMA

## 2.6 LA FASE ASSESING DE SEMMA

# FASE ASSESING (EVALUACIÓN)

El objetivo de esta fase es **determinar la utilidad real, la fiabilidad y la capacidad de generalización** del modelo.

No consiste solo en mirar un número (como un porcentaje de acierto), sino en entender **cómo se equivoca el modelo** y si esos errores son aceptables.

## Generalización vs. overfitting

La evaluación técnica busca asegurar que el modelo ha aprendido patrones, no que ha memorizado los datos de entrenamiento.

- **Overfitting (sobreajuste):** el modelo tiene un rendimiento excelente en entrenamiento, pero falla en evaluación
- **Underfitting (subajuste):** el modelo es demasiado simple y no captura la lógica ni en entrenamiento ni en evaluación.
- **Validación cruzada (K-Fold):** se utiliza en esta fase para garantizar que los resultados de la evaluación no dependan de una partición *afortunada*.

## Evaluación de problemas de clasificación

Cuando un modelo predice etiquetas, las métricas clave son:

- **Matriz de confusión**: una tabla que muestra los aciertos y errores desglosados.
- **Métricas derivadas**: de la matriz de confusión se pueden extraer datos como la precisión y la exhaustividad.
- **Curva ROC y AUC**:

**True positive:** el modelo acertó la clase positiva

**False negative:** el modelo falló la clase negativa

| | | Predicted Class: Positive | Predicted Class: Negative | |
|---|---|---|---|---|
| Actual Class | Positive | True Positive (TP) | False Negative (FN) **Type II Error** | Sensitivity $\frac{TP}{(TP+FN)}$ |
| | Negative | False Positive (FP) **Type I Error** | True Negative (TN) | Specificity $\frac{TN}{(TN+FP)}$ |
| | | Precision $\frac{TP}{(TP+FP)}$ | Negative Predictive Value $\frac{TN}{(TN+FN)}$ | Accuracy |

**False positive:** el modelo falló la clase positiva

**True negative:** el modelo acertó la clase negativa

**Predicted Class**

**Accuracy (exactitud):** es la métrica más intuitiva, pero también la más peligrosa. Indica en **qué porcentaje general acierta el modelo**.
Ejemplo: si el 99% de la población está sana y el modelo siempre dice sano, tendrá un 99% de exactitud, pero no detectará ningún enfermo

| | | |
|---|---|---|
| | egative | |
| | egative (FN) e II Error | Sensitivity $\frac{TP}{(TP+FN)}$ |
| | egative (TN) | Specificity $\frac{TN}{(TN+FP)}$ |
| $\frac{TP}{(TP+FP)}$ | e Predictive Value $\frac{TN}{(TN+FN)}$ | **Accuracy** $\frac{TP+TN}{(TP+TN+FP+FN)}$ |

**Sensibilidad (recall):** es la capacidad del modelo para detectar casos de positivos reales. Se prioriza cuando no nos podemos permitir dejar pasar un positivo (p.e. detección de cáncer)

| | | | | |
|---|---|---|---|---|
| Actual Class | Positive | True Positive (TP) | False Negative (FN) Type II Error | **Sensitivity** $\frac{TP}{(TP+FN)}$ |
| | Negative | False Positive (FP) Type I Error | True Negative (TN) | **Specificity** $\frac{TN}{(TN+FP)}$ |
| | | | | Accuracy $\frac{TP+TN}{(TN+FP+FN)}$ |

**Especifidad:** es la capacidad del algoritmo para detectar casos negativos reales. Se prioriza cuando un falso positivo es muy costoso (p.e. una operación quirúrgica que solo debe hacerse si se está seguro de una enfermedad)

**Precisión:** es la calidad de las predicciones positivas del modelo.
Se prioriza cuando queremos estar muy seguros de lo que afirmamos (p.e. al marcar un correo como spam, un error indica que el usuario pierde un email importante)

**Valor predictivo negativo (NPV):** es la calidad de las predicciones negativas del modelo.
Es fundamental para dar tranquilidad. Si un modelo da negativo en una enfermedad, el NPV dice qué probabilidad hay de que realmente no haya dicha enfermedad.

| | | | |
|---|---|---|---|
| Negative | Type I Error | True | $(TN + FP)$ |
| | **Precision** $\frac{TP}{(TP + FP)}$ | **Negative Predictive Value** $\frac{TN}{(TN + FN)}$ | **Accuracy** $\frac{TP + TN}{(TP + TN + FP + FN)}$ |

SAMPLE
MUESTREO
EXPLORE
EXPLORACIÓN
MODIFY
MODIFICACIÓN
MODEL
MODELADO
ASSES
EVALUACIÓN

RESUMEN COMPARATIVO: EL CASO DEL RADAR DE AEROPUERTO
SENSIBILIDAD ALTA
Pita por cualquier cosa metálica. Detecta todas las armas.
Muchas llaves (muchos falsos positivos). Molesta a mucha gente.
100%
SENSIBILIDAD
20%
PACIENCIA PASAJEROS
PRECISIÓN ALTA
Pita solo cuando está 100% seguro de que es un arma.
Se podrían escapar pistolas de plástico.
100%
PRECISIÓN
70%
SEGURIDAD TOTAL
ESPECIFICIDAD ALTA
SEGURIDAD IDENTIFICADA
SEGURIDAD IDENTIFICADA
Experto en identificar maletas seguras. No pita.
Si no pita, la maleta es es casi segura.
95%
ESPECIFICIDAD
NPV ALTO
PASAR
VEREDICTO RADAR:
NO ARMA
REALIDAD:
NO ARMA
PROBABILIDAD ALTÍSIMA
Si el radar te deja pasar diciendo que no llevas nada.
La probabilidad de que realmente no lleves nada prohibido es altísima.
100%
NPV

Uso `confusión_matrix` de sklearn

```python
from sklearn.metrics import confusion_matrix

# Simulo datos: 0 = Legal, 1 = Fraude
y_real = np.array([0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
                   1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1])
y_pred = np.array([0,0,0,0,1,0,0,0,1,0,0,0,0,0,0,0,0,0,0,1,0,0,0,0,0,0,
                   1,1,1,1,1,1,0,1,1,1,1,1,1,1,1,1,0,1,1,1,1,0,1,1,1])

cm = confusion_matrix(y_real, y_pred)

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Predicho: Legal', 'Predicho: Fraude'],
            yticklabels=['Real: Legal', 'Real: Fraude'])

plt.ylabel('Realidad')
plt.xlabel('Predicción')
plt.title('Matriz de Confusión: Detección de Fraude')
plt.show()
```

✓ 0.1s

En este ejemplo simulo los datos

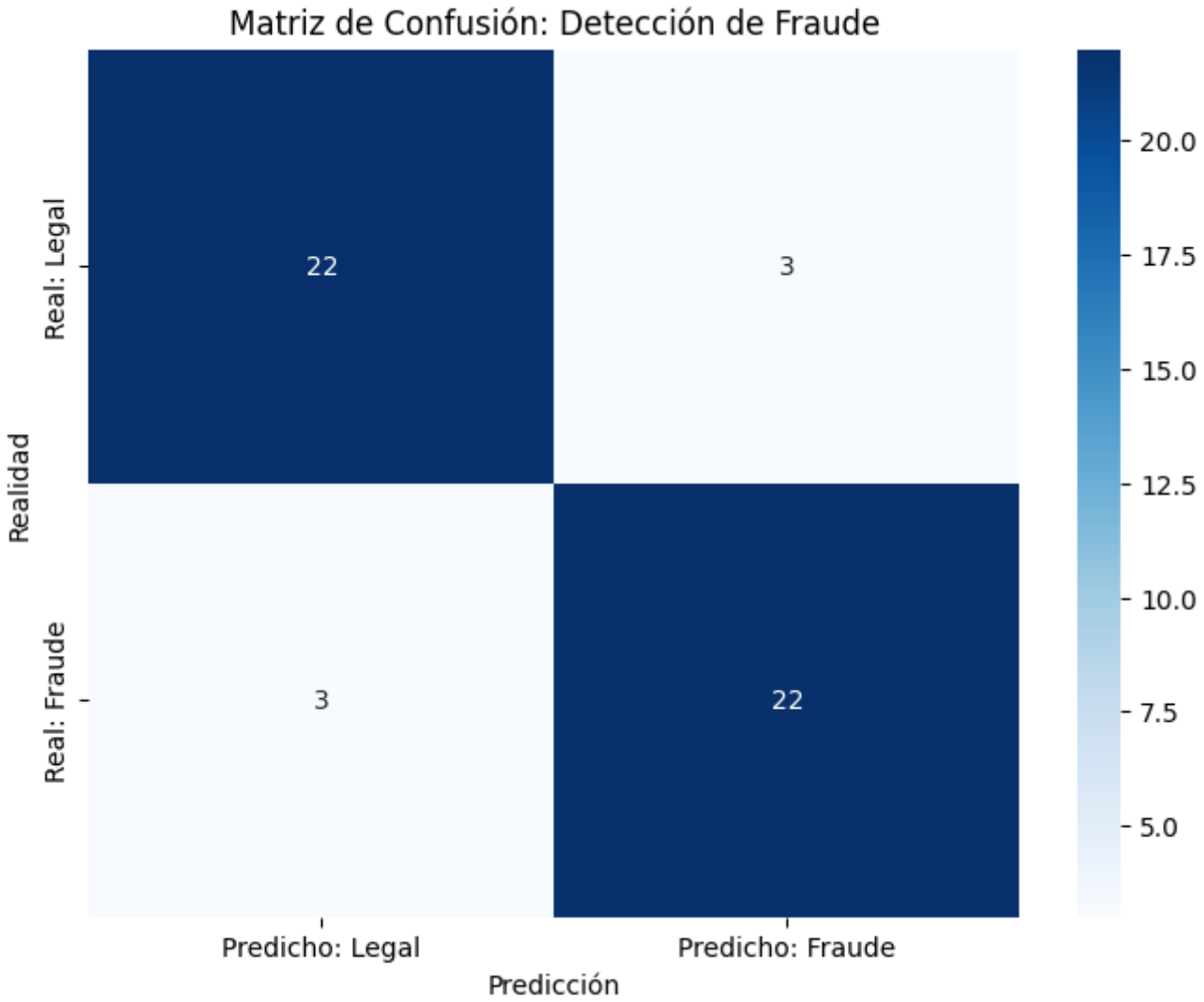
Si imprimo esta variable veré la matriz de confusión en una lista bidimensional

```
[[22  3]
 [ 3 22]]
```

```python
from sklearn.metrics import confusion_matrix

# Simulo datos: 0 = Legal, 1 = Fraude
y_real = np.array([0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
                   1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1])
y_pred = np.array([0,0,0,0,1,0,0,0,1,0,0,0,0,0,0,0,0,0,0,1,0,0,0,0,0,0,
                   1,1,1,1,1,1,0,1,1,1,1,1,1,1,1,1,0,1,1,1,1,0,1,1,1,1])

cm = confusion_matrix(y_real, y_pred)

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Predicho: Legal', 'Predicho: Fraude'],
            yticklabels=['Real: Legal', 'Real: Fraude'])

plt.ylabel('Realidad')
plt.xlabel('Predicción')
plt.title('Matriz de Confusión: Detección de Fraude')
plt.show()
```

✓ 0.1s

Para verlo más claro puedo usar Seaborn para mostrarla como un mapa de calor


Matriz de Confusión: Detección de Fraude
Realidad
Real: Legal
Real: Fraude
22
3
3
22
Predicho: Legal
Predicho: Fraude
Predicción
20.0
17.5
15.0
12.5
10.0
7.5
5.0

Para ver todos los valores asociados tenemos `classification_report`

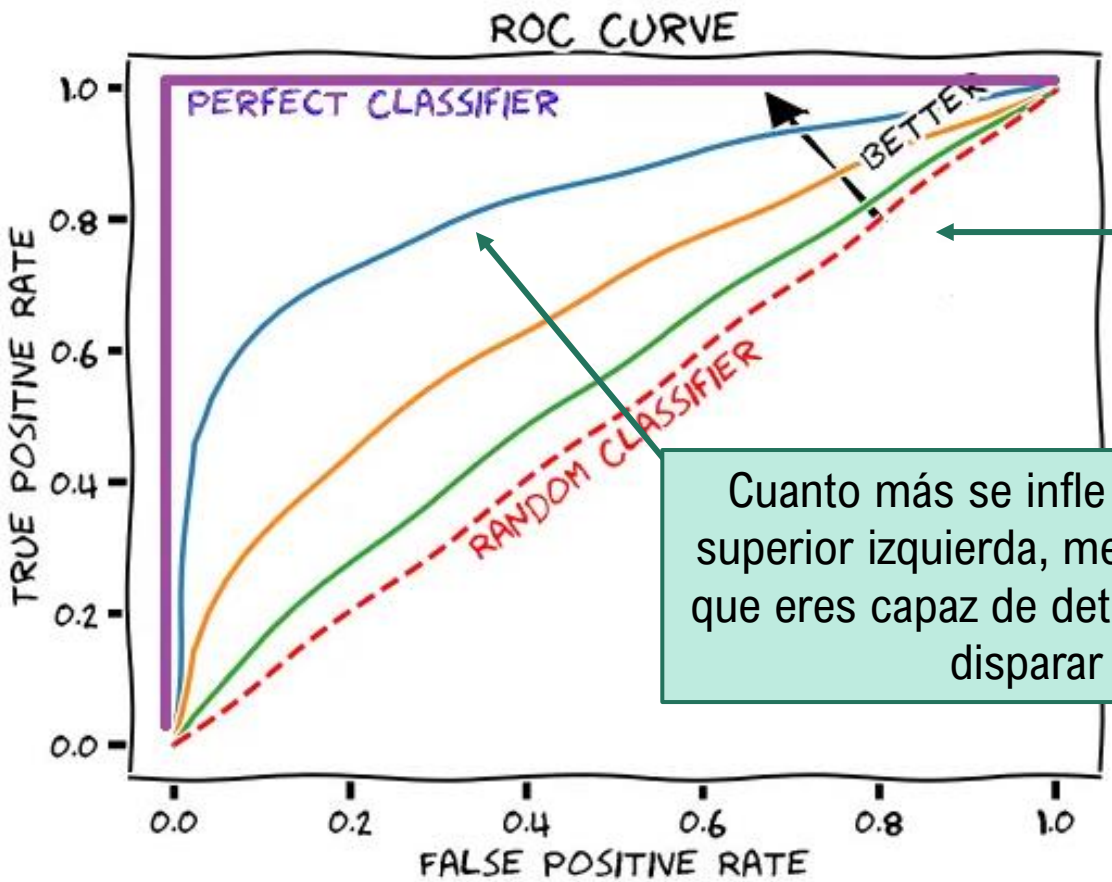
```python
from sklearn.metrics import classification_report

reporte = classification_report(y_real, y_pred, output_dict=True)
reporte
```

✓ 0.0s

```
{'0': {'precision': 0.88, 'recall': 0.88, 'f1-score': 0.88, 'support': 25.0},
 '1': {'precision': 0.88, 'recall': 0.88, 'f1-score': 0.88, 'support': 25.0},
 'accuracy': 0.88,
 'macro avg': {'precision': 0.88,
  'recall': 0.88,
  'f1-score': 0.88,
  'support': 50.0},
 'weighted avg': {'precision': 0.88,
  'recall': 0.88,
  'f1-score': 0.88,
  'support': 50.0}}
```

La **curva ROC (Receiver Operating Characteristic)** es un gráfico que muestra el rendimiento de un modelo de clasificación en **todos los umbrales de clasificación posibles**.

- En el **eje X** muestra la sensibilidad o tasa de verdaderos positivos (*de todos los positivos, ¿cuántos he acertado?*)
- En el **eje Y** muestra la tasa de falsos positivos (*de todos los negativos reales, ¿a cuántos he acusado erróneamente de ser positivos?*)




La diagonal representa un modelo que lanza una moneda al aire (azar). No tiene capacidad de discriminación

Cuanto más se infle la curva hacia la esquina superior izquierda, mejor es el modelo. Significa que eres capaz de detectar muchos positivos sin disparar casi alarmas.

El **AUC (Area Under the Curve)** es un único número que mide todo el área que queda bajo la curva:

- **Valor 0.5**: el modelo es inútil (azar)
- **Valor entre 0.7 y 0.8**: es un modelo aceptable
- **Valor entre 0.8 y 0.9:** es un modelo muy bueno
- **Valor > 0.9**: es un modelo excelente
- **Valor 1.0**: es un modelo perfecto

## Evaluación de problemas de regresión

Cuando un modelo predice números, **se mide la distancia entre la predicción y la realidad**.

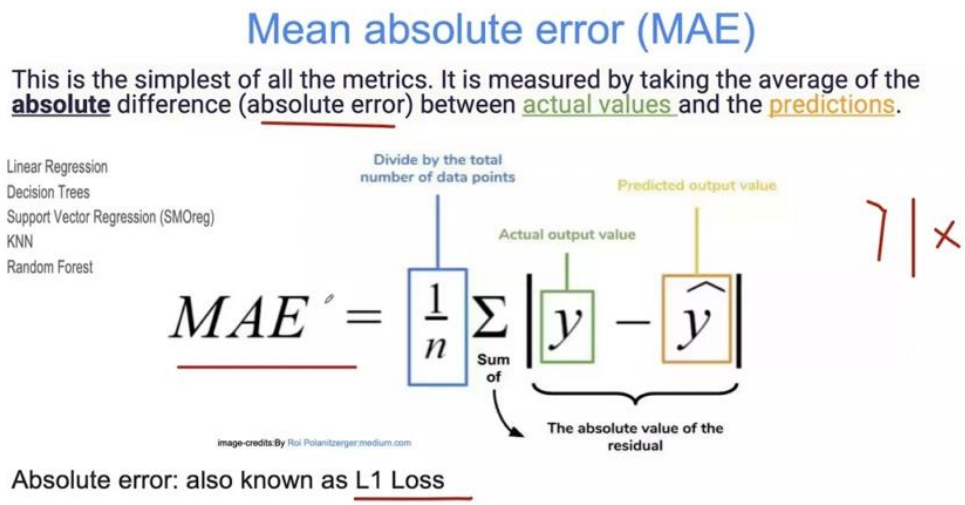
Hay diferentes formas de medirlo:

- MAE (Error absoluto medio)
- RMSE (Raíz del error cuadrático medio)

## MAE (Error absoluto medio)

La media de los errores en las mismas unidades que la variable original.

Fácil de explicar al cliente



**RMSE (Raíz del error cuadrático medio)**

Penaliza más los errores grandes. Es la métrica estándar para optimización matemática.

$$RECM = \sqrt{\frac{\sum(O_i - E_i)^2}{n}}$$

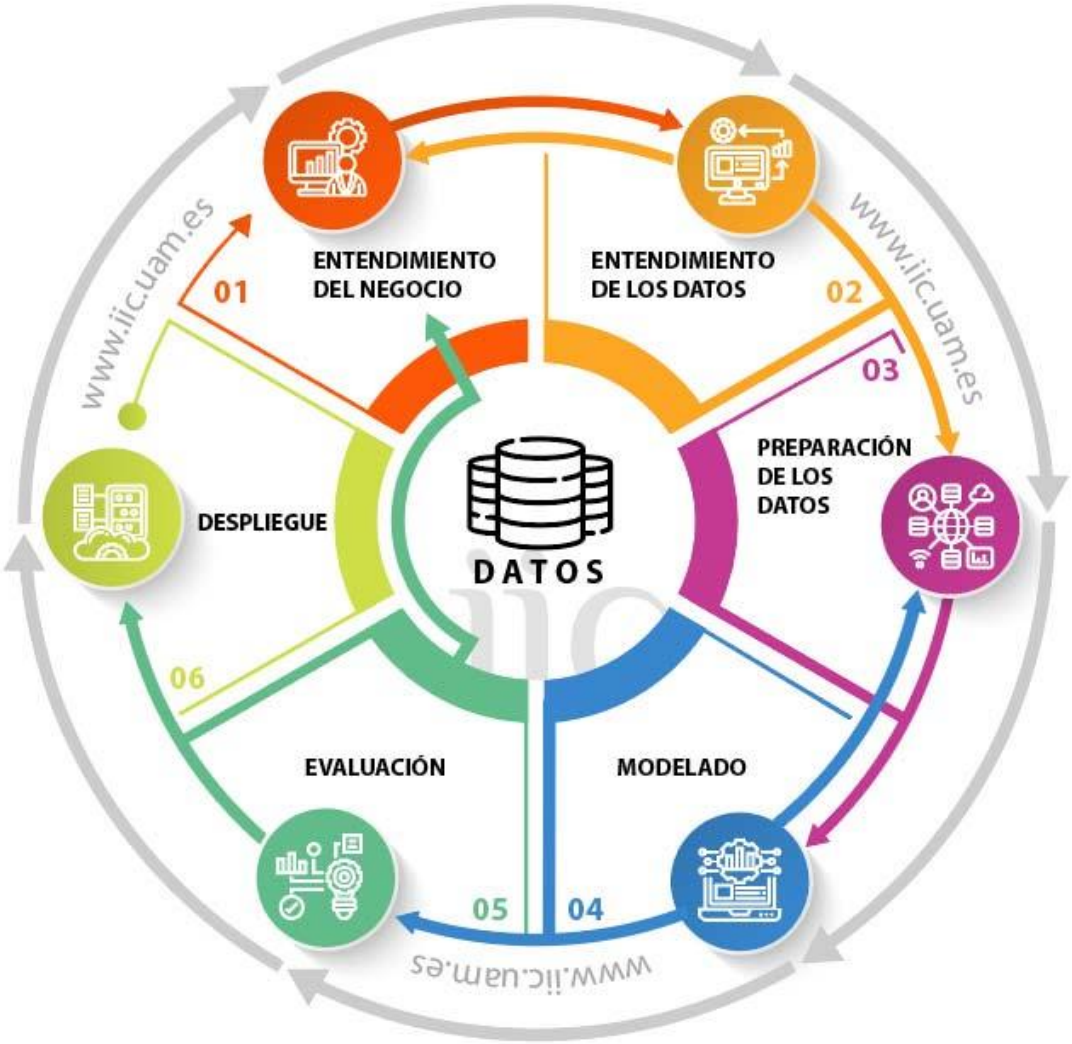
# 3 LA METODOLOGÍA CRISP-DM

# 3 LA METODOLOGÍA CRISP-DM

## 2.6 INTRODUCCIÓN A LA METODOLOGÍA CRISP-DM

Si SEMMA es el enfoque en entornos académicos, **CRISP-DM (Cross-Industry Standard Process for Data Mining)** es el enfoque tomado en entornos empresariales.

Es la metodología más usada en el mundo real porque no empieza con los datos, sino con el dinero y los objetivos de negocio.



# 3 LA METODOLOGÍA CRISP-DM

## 3.2 FASE DE COMPRENSIÓN DE NEGOCIO

# FASE COMPRENSIÓN DE NEGOCIO

Es la fase de traducción. El objetivo es convertir las necesidades de una empresa en un problema de ciencia de datos.

Las tareas clave de esta fase son:

- **Determinar objetivos**: ¿qué quiere la empresa?
- **Evaluar la situación**: recursos disponibles, riesgos y análisis de costes-beneficios.
- **Definir objetivos de minería de datos**: ¿Qué métrica técnica define el éxito? (p.e. lograr un RMSE menor a 10 unidades)
- **Producir el plan de proyecto**: cronograma y herramientas

# El entregable en esta fase debería contener lo siguiente:

| Bloque | Fase | Descripción |
|---|---|---|
| 1. Objetivo de negocio | Negocio | ¿Qué problema queremos resolver? |
| 2. Valor del proyecto | Negocio | ¿Cuánto dinero esperamos ganar o ahorrar? ¿Cuál es el retorno de la inversión? |
| 3. Fuentes de datos | Datos | ¿De dónde sacamos la información? (ERP, APIs, Web Scraping, Logs) |
| 4. Variable objetivo | Datos | ¿Qué queremos predecir exactamente? (p.e. una cantidad numérica, una categoría) |
| 5. Calidad y riesgos | Datos | ¿Faltan datos? ¿Están sesgados?¿Hay problemas legales con estos datos? |
| 6. Preparación y features | Preparación | ¿Qué transformaciones clave haremos? (Escalados, codificación, creación de KPIs) |
| 7. Modelado | Modelado | ¿Qué tipo de algoritmos vamos a probar? (random forest, redes neuronales, …) |
| 8. Evaluación (KPIs) | Evaluación | Técnico: R2, AUC, F1 \| Negocio: % de acierto en pedidos, ahorro |
| 9. Despliegue y uso | Despliegue | ¿Cómo llegará la predicción al usuario? (Excel semanal, API, dashboard) |

# Ejemplo práctico: **Predicción de abandono escolar**

| Bloque | Descripción |
|---|---|
| **1. Objetivo de negocio** | Reducir la tasa de abandono en la ESO en un 15% durante el presente curso escolar. |
| **2. Valor del proyecto** | **Social**: mejora de la empleabilidad futura del alumnado,<br>**Económico**: optimización de costes por ratio de alumnos y reducción de costes operativos por re-matriculaciones. |
| **3. Fuentes de datos** | Sistema de gestión académica (notas finales y parciales).<br>Registros de asistencia diaria.<br>Datos sociodemográficos (distancia al centro, nivel de estudios de los padres)<br>Registro de becas |
| **4. Variable objetivo** | **Binaria**: abandona (1) o continua (0)<br>Se define como abandono si el alumno deja de asistir formalmente o se desvincula antes de finalizar el segundo trimestre. |
| **5. Calidad y riesgos** | **Calidad**: faltas de asistencia suelen estar incompletas<br>**Riesgos**: sesgo algorítmico (etiquetar alumnos por su nivel de renta) y cumplimiento estricto de la RGDP al tratarse de menores. |

| Bloque | Descripción |
|---|---|
| **6. Preparación y features** | Creación de la variable “Pendiente de notas” (evolución entre parciales)<br>Cálculo del % de faltas injustificadas de los últimos 30 días<br>Binarización de la variable *hermanos en el mismo centro*. |
| **7. Modelado** | Algoritmos de clasificación supervisada: regresión logística y random forest. |
| **8. Evaluación (KPIs)** | **Técnico**: priorizar la sensibilidad (recall) > 0.85 para no dejar ningún alumno en riesgo fuera<br>**Negocio**: tasa de éxito de las tutorías de intervención realizadas. |
| **9. Despliegue y uso** | Un **dashboard interactivo** para el departamento de orientación y una **alerta automática por mail** al tutor cuando la probabilidad de abandono de un alumno supere el 70% |

# 3 LA METODOLOGÍA CRISP-DM

## 3.3 FASE DE COMPRENSIÓN DE LOS DATOS

# FASE COMPRENSIÓN DE LOS DATOS

Esta fase consiste en analizar los datos que tenemos y cuestionarlos para saber si son capaces de responder a nuestras preguntas.

Tiene 5 tareas técnicas fundamentales:

- Recolección inicial de los datos
- Descripción de los datos
- Exploración de los datos (EDA – Exploratory Data Analysis)
- Verificación de la calidad de los datos

## Recolección inicial de los datos

Es el primer contacto técnico.

Debemos **documentar de dónde vienen los datos** y cómo han llegado a nuestras manos.

- **Inventario de fuentes**: bases de datos relacionales, archivos planos (CSV, JSON), logs de servidores, servicios externos (APIs), …
- **Muestreo inicial**: si el volumen de datos es **masivo**, seleccionar una muestra representativa para esta fase de exploración inicial.
- **Documentación de carga**: anotar si ha habido problemas en la transferencia (caracteres extraños, truncado de textos, …)

## Descripción de los datos

Definimos la estructura de los datos a nivel superficial:

- **Análisis de metadatos**: ¿qué significa cada columna?
- **Tipología de variables**:
  - ***Numéricas***: discretas (conteo) o continuas (medición)
  - ***Categóricas***: nominales (sin orden, p.e. color) u ordinales (con orden, p.e. nivel de estudios)
- **Volumen**: número de registros (filas) y de características (columnas). Esto nos dará una idea de la complejidad computacional a la que no vamos a enfrentar.

## Exploración de los datos (EDA – Exploratory Data Analysis)

Buscamos **patrones y relaciones ocultas** mediante estadística descriptiva y visualización. La idea es dejar de ver *tablas* para ver *comportamientos*.

Los **objetivos principales** de esta fase son:

- Comprender la distribución
- Identificar relaciones
- Detectar anomalías

Tres **niveles de complejidad**:

- Análisis univariante
- Análisis bivariante y multivariante
- Análisis de series temporales

## EDA: Análisis univariante

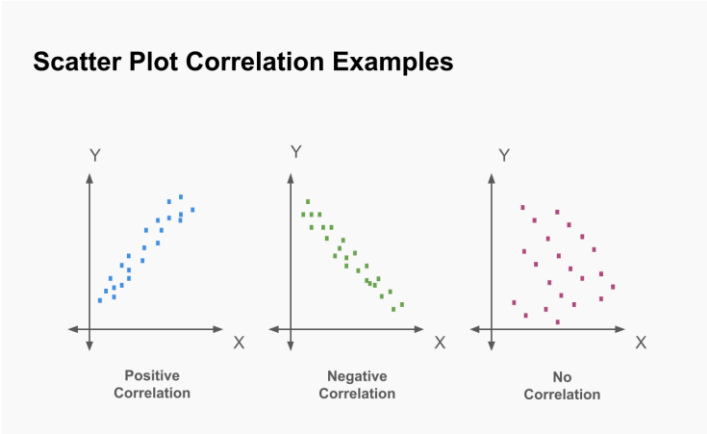
Se analiza cada columna para entender su forma y rango.

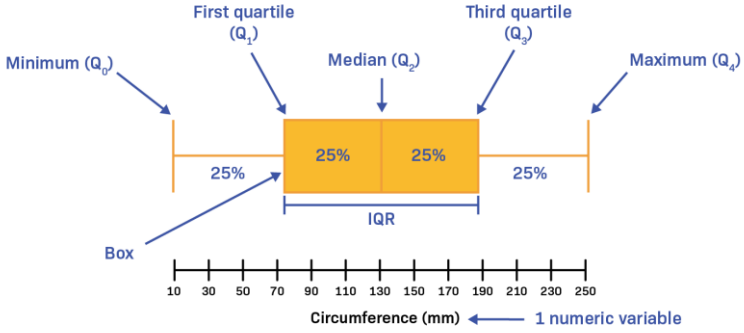
- **Estadística descriptiva**: uso de `.describe()` para obtener la media, mediana, desviación típica y cuartiles.
- **Visualización de distribuciones**:
  - Histograma y KDE para ver si los datos siguen una distribución normal o están sesgados.
  - Boxplots (diagramas de caja): para ver la dispersión y los outliers
- **Conteo de categorías**: para variables cualitativas (p.e. tipos de edificios), ver si las clases están equilibradas

# EDA: Análisis bivariante y multivariante

Buscamos cómo se interrelacionan las variables

- **Matrices de correlación (heatmaps)**
- **Scatter plots** (gráficos de dispersión)
- **Boxplots agrupados**: compara una variable numérica frente a una categórica



## EDA: Análisis de series temporales (temporalidad)

El objetivo es identificar patrones que dependen exclusivamente del paso del tiempo.

Algunas visualizaciones comunes son:

- Gráficos de líneas
- Gráficos de autocorrelación (ACF/PACF)

# 3 LA METODOLOGÍA CRISP-DM

## 3.4 FASE DE PREPARACIÓN DE LOS DATOS

# FASE PREPARACIÓN DE LOS DATOS

Esta fase es la que más tiempo consume en cualquier proyecto.

Su objetivo es **transformar los datos en bruto en el dataset final** que se introducirá en los algoritmos de modelado.

Un error aquí invalida cualquier resultado posterior (**GIGO** – Garbage In, Garbage Out)

Las tareas que componen esta fase son:

- Selección de datos
- Limpieza de datos
- Construcción de datos / Feature engineering
- Integración de datos
- Formateo de datos

## Selección de datos

No todos los datos disponibles son útiles o éticos para el modelo.

Algunas tareas a realizar son:

- **Selección de variables (columnas):** se eliminan columnas con demasiados valores nulos, variables con varianza 0 o aquellas que no tienen relación lógica con el modelo.
- **Selección de registros (filas):** se filtran subpoblaciones específicas o se eliminan registros que son errores de medición confirmados
- **Muestreo**: si el volumen es inabarcable, se decide una estrategia de muestreo (aleatorio, estratificado, …)

## Limpieza de datos

Es la tarea más técnica y rigurosa. Consiste en **elevar la calidad** de los datos seleccionados.

Algunas tareas a realizar son:

- **Tratamiento de nulos (imputación):**
  - Eliminar registros si son pocos
  - Imputar valores estadísticos (media, mediana, moda)
  - Uso de modelos predictivos para estimar el valor faltante
- **Tratamiento de outliers**: decidir si se eliminan, se transforman o se mantienen.
- **Corrección de inconsistencias**: unificar formatos (“M”, “Masculino”, “H” -> Hombre)

## Construcción de datos / Feature engineering

Se crean **nuevas variables** para ayudar al modelo a entender mejor el patrón.

Por ejemplo:

- **Derivación de atributos**: a partir de una fecha de compra crear `día_de_la_semana` o `es_fin_de_semana`.
- **Agregaciones**: convertir datos de grano fino a grano grueso (p.e. de ventas diarias a ventas mensuales)
- **Transformaciones matemáticas**: aplicar logaritmos para normalizar distribuciones sesgadas.

## Integración de datos

En esta tarea **se combinan datos procedentes de diferentes fuentes**.

Normalmente consiste en unir tablas relacionales mediante claves comunes con **joins** y **merges**.

Hay que asegurarse de que al unir diferentes tablas las unidades de medida sean las mismas (p.e. no mezclar euros con dólares sin conversión)

## Formateo de datos

Consiste en realizar cambios estéticos o de estructura que no cambian el significado del dato, pero son necesarios para los algoritmos.

Ejemplos:

- **Codificación de variables categóricas**: one-hot-encoding o label encoding.
- **Escalado y normalización**: asegurar que todas las variables numéricas estén en el mismo rango:
  - Min-max scaling
  - Estandarización (Z-Score)

# 3 LA METODOLOGÍA CRISP-DM

## 3.5 FASE DE MODELADO

# FASE MODELADO DE LOS DATOS

Aquí se aplican los algoritmos matemáticos y estadísticos a los datos para encontrar patrones, realizar predicciones o descubrir agrupaciones.

Es un proceso **experimental** y **altamente iterativo** que se divide en cuatro tareas:

1. Seleccionar la técnica de modelado
2. Generar el plan de prueba
3. Construcción del modelo
4. Evaluación técnica del modelo

## Selección de la técnica de modelado

Se elige la técnica de modelado según el objetivo de negocio definido en la primera fase.

**Tipo de problema**:

- **Clasificación**: para predecir etiquetas (Fraude/No fraude)
- **Regresión**: para predecir valores numéricos (precio de la luz)
- **Clustering**: para agrupar datos sin una etiqueta previa.

## Generar el plan de prueba

Para saber si el modelo funciona hay que escoger una técnica de validación:

- **División train/test**: p.e. 80% de los datos para entrenar y 20% para test
- **Validación cruzada (K-Fold Cross Validation):** el dataset se divide en k partes y el modelo se entrena k veces utilizando diferentes combinaciones

## Construcción del modelo

Es el entrenamiento propiamente dicho.

Se pueden buscar diferentes combinaciones de hiperparámetros para encontrar la combinación óptima.

# Evaluación técnica del modelo

Se utilizan diferentes métricas para evaluar el rendimiento del modelo.

En esta fase evaluamos si el modelo **es bueno matemáticamente**.

# 3 LA METODOLOGÍA CRISP-DM

## 3.6 FASE DE EVALUACIÓN

# FASE EVALUACIÓN

En la última tarea de la fase anterior evaluamos si el modelo es **preciso**, en esta fase **evaluamos si es útil para el negocio**.

Hay tres tareas en esta fase:

- Evaluación de resultados de cara al negocio
- Revisión del proceso
- Determinación de los siguientes pasos

## Evaluación de resultados de cara al negocio

El objetivo es comprobar si el modelo construido realmente resuelve los objetivos planteados en la primera fase.

**Ya no hablamos de RMSE o F1-Score, sino de dinero, tiempo o eficiencia.**

Ejemplo: "*Con este modelo, el ayuntamiento ahorrará un 12% en su factura eléctrica mensual*"

## Revisión del proceso

Antes de dar paso al despliegue, se realiza una **auditoría** de todo lo que se ha hecho en todas las fases para asegurar que no haya errores ocultos.

*¿La muestra de datos fue representativa? ¿Se documentaron correctamente las transformaciones? ¿Se han usado accidentalmente datos del futuro para predecir el pasado?*

## Determinación de los siguientes pasos

En base a los resultados anteriores, **se toma una decisión**:

- Se aprueba el **despliegue**
- **Iteración**. Se ha detectado que faltan datos o los objetivos iniciales han cambiado. Se vuelve a una fase anterior.
- **Cancelación**: los datos no tienen suficiente poder predictivo o el coste de implementar el modelo es mayor que el beneficio que aporta.

# 3 LA METODOLOGÍA CRISP-DM

## 3.7 FASE DE DESPLIEGUE

# FASE DESPLIEGUE

Hasta aquí todo se ha hecho sobre papel o en un entorno de desarrollo con Jupyter o similar.

Ahora, es el momento de integrar el modelo en los procesos de la empresa para que empiece a generar valor.

## Planificación del despliegue

Se define cómo se va a utilizar el modelo en el día a día.

Dos enfoques:

- **Batch**: el modelo se ejecuta, por ejemplo, una vez a la semana y genera un reporte o actualiza una base de datos
- **Real-time**: el modelo está conectado a una aplicación (vía API) y da una respuesta inmediata cada vez que se introduce un dato.

**¿Cómo se integrará?:** Dashboard de Power BI, App móvil, software interno de la empresa

## Planificación de la monitorización y el mantenimiento

Los modelos de IA tienen fecha de caducidad porque el mundo (y los datos) cambian. A esto se le llama **model drift** o **deriva del modelo**.

Debemos crear **alertas** para saber si el modelo empieza a fallar (por ejemplo, el precio de la luz sube tanto que el modelo no tiene sentido)

También hay que definir **cada cuánto tiempo se va a re-entrenar el modelo** con datos nuevos para que no quede obsoleto.

## Producción del informe final

Se crea la **documentación técnica y de negocio** que resume todo el proyecto.